Contents

1 Proyecto Big Data KDD **5**
1.1 Entrega Unificada . . . 5

2 Paquete Final de Entrega **6**
2.1 Portada . . . 6
2.2 1. Documentos principales . . . 6
2.3 2. Alcance funcional entregado . . . 6
2.4 3. Checklist de revision previa a entrega . . . 7
2.4.1 3.1 Infraestructura . . . 7
2.4.2 3.2 Datos y pipeline . . . 7
2.4.3 3.3 Dashboard . . . 8
2.4.4 3.4 Documentacion . . . 8
2.5 4. Comandos de validacion rapida . . . 8
2.6 5. Generacion de PDFs de entrega . . . 8
2.7 6. Nota final . . . 9

3 Resumen Ejecutivo - Sistema KDD Logistico **9**
3.1 Portada . . . 9
3.2 Indice . . . 9
3.3 1. Que se ha construido . . . 9
3.4 2. Arquitectura y flujo de valor . . . 10
3.4.1 Ingesta . . . 10
3.4.2 Procesamiento . . . 10
3.4.3 Almacenamiento y consulta . . . 10
3.4.4 Explotacion . . . 10
3.5 3. Resultados funcionales alcanzados . . . 11
3.6 4. Robustez y operaciones . . . 11
3.7 5. Estado tecnico final (entrega) . . . 11
3.8 6. Entregables documentales . . . 12
3.9 7. Conclusiones . . . 12

4 Memoria Tecnica del Sistema KDD Logistico **12**
4.1 Portada . . . 12
4.2 Indice . . . 12
4.3 1. Objetivo del sistema . . . 13
4.4 2. Arquitectura general . . . 13
4.4.1 Diagrama de arquitectura . . . 14
4.5 3. Flujo de datos end-to-end . . . 14
4.5.1 3.1 GPS . . . 14
4.5.2 3.2 Meteorologia . . . 14
4.5.3 3.3 Batch historico . . . 15
4.6 4. NiFi en detalle . . . 15
4.7 4.1 Estructura visual desplegada . . . 15
4.8 4.2 Controller Service . . . 16
4.9 4.3 Flujo GPS: procesadores y rol . . . 16
4.10 4.4 Flujo Clima: procesadores y rol . . . 17
4.11 4.5 Back-pressure y terminacion . . . 18
4.12 5. Generacion y sincronizacion raw . . . 18

4.13 5.1 GPS generator . . . 18
4.14 5.2 Raw HDFS loader . . . 18
4.15 6. Spark en detalle . . . 18
4.16 6.1 Preparacion comun . . . 19
4.17 6.2 Pipeline batch . . . 19
4.18 6.3 Pipeline streaming . . . 20
4.19 6.4 ML de riesgo de retraso . . . 21
4.20 7. Hive: tablas y vistas . . . 22
4.21 7.1 Tablas maestras y batch . . . 22
4.22 7.2 Tablas streaming . . . 22
4.23 7.3 Vistas en hora Madrid . . . 22
4.24 8. Cassandra . . . 22
4.25 9. Airflow en detalle . . . 23
4.26 10. Dashboard . . . 24
4.27 11. Orquestacion de arranque y reset . . . 26
4.28 12. Operacion y validacion recomendada . . . 26
4.29 13. Puntos de resiliencia implementados . . . 27
4.30 14. Limitaciones conocidas . . . 27

5 Flujo NiFi sugerido **27**
5.1 Portada . . . 27
5.2 Objetivo . . . 28
5.3 Flujo minimo (GPS) . . . 28
5.4 Recomendaciones . . . 29
5.5 Ingesta API meteorologica (requisito del enunciado) . . . 29
5.6 Generador GPS para NiFi . . . 30
5.7 Bootstrap automatico NiFi . . . 30

6 Dashboard Logistico **31**
6.1 Portada . . . 31
6.2 Indice . . . 31
6.3 Objetivo . . . 31
6.4 Captura actual . . . 31
6.5 Fuentes de datos del dashboard . . . 31
6.6 Endpoints operativos del dashboard . . . 33
6.7 Que representa cada elemento visual . . . 33
6.7.1 Mapa en tiempo real . . . 33
6.7.2 Filtros de Tiempo Real (solo vista izquierda) . . . 34
6.7.3 Mapa de red logistica . . . 34
6.7.4 Tabla de ruta (debajo del mapa logistico) . . . 34
6.7.5 Insights de red (live) . . . 35
6.8 Consolidacion en Hive (reporting) . . . 35
6.9 Grafo por cercania geografica (k vecinos) . . . 35
6.9.1 Filtros de Analisis Logistico (solo vista derecha) . . . 36
6.10 Reentreno IA (estado de modelo) . . . 36
6.11 Concurrencia de reentreno (Airflow + Dashboard) . . . 37
6.12 Reglas de interpretacion de ruta por vehiculo . . . 37
6.13 ETA en panel de vehiculo . . . 38
6.14 Datos y nodos actuales . . . 38

6.15 Reinicio limpio de demo . . . 38
6.16 Nota de uso . . . 39

7 Operaciones del Proyecto KDD **39**
7.1 Portada . . . 39
7.2 Indice . . . 39
7.3 Arranque y parada . . . 40
7.4 Comandos operativos rapidos (scripts clave) . . . 40
7.4.1 Reinicio completo o parcial de servicios . . . 40
7.4.2 Checks de salud de la plataforma . . . 40
7.4.3 Comprobaciones y validaciones por script . . . 41
7.4.4 Consultas BBDD (Hive y Cassandra) . . . 41
7.4.5 Troubleshooting rapido . . . 41
7.5 Healthcheck en Airflow . . . 42
7.6 Arranque del stack desde Airflow . . . 43
7.7 Flujo NiFi . . . 43
7.8 Validaciones . . . 43
7.9 Politica de almacenamiento . . . 44
7.10 Consultas desde terminal (Hive y Cassandra) . . . 44
7.10.1 Hive (via spark-sql en `spark-client`) . . . 44
7.10.2 Cassandra (via `cqlsh` en contenedor `cassandra`) . . . 45
7.11 Sync de insights Cassandra -> Hive . . . 45
7.12 Matriz operativa end-to-end . . . 46
7.13 Limpieza de estado streaming . . . 47
7.14 Limpieza de historico failed en Airflow . . . 47
7.15 Zona horaria . . . 48
7.16 Vistas Hive en hora Madrid . . . 48
7.17 Bitacora operativa (actualizada 04/04/2026) . . . 48
7.18 Nota de tablas en tiempo real . . . 48
7.19 Operacion del dashboard (estado final) . . . 49
7.19.1 Desacople de vistas . . . 49
7.19.2 Reglas de filtro en Tiempo Real . . . 49
7.19.3 Fuente de ruta por vehiculo . . . 49
7.20 Troubleshooting dashboard . . . 50
7.20.1 Caso: veo ruta incoherente (`X -> X`) o destino inesperado . . . 50
7.20.2 Caso: ETA incoherente (ejemplo: >300 km y ETA en pocos minutos) . . . 50
7.21 Reentreno IA del dashboard . . . 50
7.21.1 Endpoints . . . 51
7.21.2 Variables de entorno relevantes . . . 51
7.21.3 Persistencia Cassandra . . . 51
7.21.4 Concurrencia y bloqueo de triggers . . . 51
7.22 Preview HDFS en navegador (temporal y riesgos) . . . 52

8 Manual de Usuario - Plataforma KDD Logistica **52**
8.1 Portada . . . 52
8.2 Indice . . . 52
8.3 1. Objetivo y alcance . . . 53
8.4 2. Requisitos previos . . . 53
8.5 3. Arranque y parada del stack . . . 53

8.5.1 3.1 Arranque recomendado (todo en uno) . . . 53
8.5.2 3.2 Arranque alternativo (Airflow primero) . . . 54
8.5.3 3.3 Parada completa . . . 54
8.5.4 3.4 Reinicio rapido por servicio . . . 54
8.5.5 3.5 Hard reset de demo . . . 54
8.6 4. Uso completo del dashboard . . . 54
8.6.1 4.1 Estructura funcional del dashboard . . . 54
8.6.2 4.2 Vista de Tiempo Real (izquierda) . . . 55
8.6.3 4.3 Vista de Red Logistica (derecha) . . . 55
8.6.4 4.4 Flujo recomendado de uso del dashboard . . . 56
8.6.5 4.5 Diagnostico de fuentes del dashboard . . . 56
8.6.6 4.6 Reentreno IA desde dashboard . . . 56
8.7 5. Uso de DAGs en Airflow . . . 56
8.7.1 5.1 DAG `kdd_bootstrap_stack` . . . 57
8.7.2 5.2 DAG `kdd_hourly_healthcheck` . . . 57
8.7.3 5.3 DAG `kdd_daily_healthcheck` . . . 57
8.7.4 5.4 DAG `logistics_kdd_monthly_maintenance` . . . 58
8.7.5 5.5 Como ejecutar DAGs manualmente . . . 58
8.8 6. Comprobaciones de salud y estado . . . 58
8.8.1 6.1 Estado de servicios . . . 58
8.8.2 6.2 Estado de topics Kafka . . . 58
8.8.3 6.3 Estado de HDFS . . . 58
8.8.4 6.4 Estado de objetos Hive . . . 59
8.9 7. Consultas de datos (Hive y Cassandra) . . . 59
8.9.1 7.1 Consultas Hive utiles . . . 59
8.9.2 7.2 Consultas Cassandra utiles . . . 59
8.10 8. Troubleshooting guiado . . . 59
8.10.1 8.1 Dashboard no responde . . . 59
8.10.2 8.2 Vista meteo vacia en Hive (`weather_rows=0`) . . . 59
8.10.3 8.3 Estado streaming inconsistente . . . 59
8.10.4 8.4 Flujo NiFi desactualizado o ausente . . . 60
8.10.5 8.5 Runs `failed` antiguos en Airflow . . . 60
8.10.6 8.6 ETA/ruta incoherente en frontend . . . 60
8.10.7 8.7 HDFS UI muestra “Couldn’t preview the file” . . . 60
8.11 9. Comandos de referencia rapida . . . 60

9 Arquitectura del Proyecto **61**
9.1 Portada . . . 61
9.2 Indice . . . 61
9.3 1. Vista general . . . 61
9.4 2. Diagrama de arquitectura . . . 62
9.5 3. Componentes y responsabilidades . . . 62
9.6 4. Flujo de datos end-to-end . . . 64
9.7 5. Capas de almacenamiento . . . 64
9.8 6. Decision de desacople en dashboard . . . 64

10 Release Notes - 04/04/2026 **64**
10.1 Portada . . . 64
10.2 Resumen ejecutivo . . . 64

10.3 Cambios funcionales . . . 65
10.3.1 Airflow + DAG mensual . . . 65
10.3.2 Dashboard (reentreno IA) . . . 65
10.3.3 HDFS UI (preview de fichero) . . . 65
10.4 Cambios de documentacion . . . 65
10.5 Implicaciones operativas (HDFS preview) . . . 66
10.6 Referencias rapidas . . . 66

11 Proyecto Big Data KDD - Java 21 + Docker 66
11.1 Arquitectura . . . 66
11.2 Estructura . . . 67
11.3 Arranque rapido . . . 67
11.4 Scripts operativos . . . 68
11.5 Operacion diaria (checks, reinicios, BBDD y troubleshooting) . . . 69
11.6 Validacion automatica Hive . . . 70
11.7 Formato de datos (raw vs procesado) . . . 70
11.7.1 Matriz operativa (resumen) . . . 71
11.8 Vistas Hive en hora Madrid . . . 71
11.9 Servicios . . . 72
11.10 Cambios recientes (03/04/2026) . . . 73
11.11 Cambios recientes (04/04/2026) . . . 74
11.12 Cambios recientes (31/03/2026) . . . 74
11.13 Cambios recientes (30/03/2026) . . . 75
11.14 Notas . . . 76

12 Changelog 76
12.1 [Unreleased] . . . 76
12.1.1 Added . . . 76
12.1.2 Changed . . . 77
12.1.3 Fixed . . . 78
12.1.4 Removed . . . 78
12.2 [1.0.0] - 2026-03-30 . . . 78
12.2.1 Added . . . 78
12.2.2 Changed . . . 78
12.2.3 Fixed . . . 79
12.2.4 Removed . . . 79
12.2.5 Documentation . . . 79

1 Proyecto Big Data KDD

1.1 Entrega Unificada

Autor: Rául Meneses Gutiérrez

Curso: Curso de Especialización en Inteligencia Artificial y BigData

Fecha: 04/04/2026

Repositorio: https://github.com/raulsistemasydesarrollo/KDD-Ingenieria-de-datos

2 Paquete Final de Entrega

2.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Indice maestro de entrega`
- Version: `v1.4`
- Fecha: `04/04/2026`
- Repositorio GitHub: `https://github.com/raulsistemasydesarrollo/KDD-Ingenieria-de-datos`

2.2 1. Documentos principales

1. Memoria tecnica completa:
   - docs/memoria-tecnica-sistema.md
2. Resumen ejecutivo (1-2 paginas):
   - docs/resumen-ejecutivo-memoria.md
3. Guia funcional del dashboard:
   - docs/dashboard.md
4. Guia de operaciones y troubleshooting:
   - docs/operations.md
5. Manual de usuario integral (arranque/parada, dashboard, Airflow DAGs y uso diario):
   - docs/manual-usuario.md
6. Release notes de la iteracion:
   - docs/release-notes-2026-04-04.md
7. Guia general del repositorio:
   - README.md
8. Arquitectura tecnica + diagrama:
   - docs/architecture.md
   - docs/architecture-diagram.png
9. Guia de anotacion de codigo:
   - docs/CODE-ANNOTATION-GUIDE.md
10. Changelog consolidado:

- CHANGELOG.md

2.3 2. Alcance funcional entregado

1. Plataforma KDD end-to-end con NiFi, Kafka, Spark, Hive, Cassandra, Airflow y dashboard web.
2. Ingesta separada por dominios (`gps_ingestion` y `weather_ingestion`) en NiFi.
   - PG por defecto actual: `kdd_ingestion_auto_v9`.
   - Archivado GPS sin sobrescritura por split (filename unico por evento).
3. Dashboard con dos vistas desacopladas:
   - Tiempo Real con filtros propios (`Origen RT`, `Destino RT`).
   - Red Logistica con filtros propios (`Origen`, `Destino`, `Perfil`).
4. Routing logistico multiobjetivo:

- perfiles `balanced`, `fastest`, `resilient`, `eco`, `low_risk`, `reliable`,
- pesos `tiempo/riesgo/eco`,
- modo temporal (`auto`, `peak`, `offpeak`, `night`),
- exclusion de nodos intermedios.

5. Rutas y proyecciones de vehiculo coherentes con plan de trayecto (`planned_origin`, `planned_destination`, `planned_route_nodes`, `planned_route_label`).
6. Estado operativo con fuentes primarias en Cassandra y fallback controlado.
7. Insights de red live con historico persistido en Cassandra y consolidacion en Hive.
8. Operativa de reentreno IA en dashboard:
   - `POST /api/ml/retrain`,
   - `GET /api/ml/retrain/status`,
   - estado persistido en `transport.model_retrain_state`,
   - panel de cabecera con modelo en uso + candidatos y criterio de seleccion.
9. Red geografica consolidada (15 nodos) y flota ampliada (15 vehiculos).
10. Entrenamiento ML robustecido con comparativa automatica entre `baseline_rf`, `tuned_baseline_rf` y `enhanced_rf`, seleccionando el menor RMSE en cada batch/reentreno.
    - `enhanced_rf` incorpora clima y congestion alineados temporalmente (ventanas de 15 minutos) y es el seleccionado en la validacion final de esta iteracion.
11. Documentacion visual actualizada con capturas operativas vigentes:
    - `docs/airflow.png`
    - `docs/dashboard.png`
12. Preview HDFS desde navegador documentado con opcion segura (alias host) y opcion temporal con riesgos operativos.

2.4 3. Checklist de revision previa a entrega

2.4.1 3.1 Infraestructura

- ☐ `docker compose ps` sin servicios core caidos.
- ☐ NiFi accesible en `https://localhost:8443/nifi`.
- ☐ Airflow accesible en `http://localhost:8080`.
- ☐ Dashboard accesible en `http://localhost:8501`.

2.4.2 3.2 Datos y pipeline

- ☐ Topic Kafka GPS/Weather creados y activos.
- ☐ Raw en HDFS presente (`/data/raw/nifi/...`).
- ☐ Tablas Hive principales visibles en `transport_analytics`.
- ☐ Tabla de eventos enriquecidos streaming visible y con datos:
  - `transport_analytics.enriched_events_streaming`.
- ☐ Cassandra con estado de flota y observaciones meteo recientes.
- ☐ Tabla Cassandra de snapshots de insights con datos:
  - `transport.network_insights_snapshots`.
- ☐ Tablas Hive de insights consolidadas con datos:

- `transport_analytics.network_insights_snapshots_hive`
- `transport_analytics.network_insights_hourly_trends`.

2.4.3 3.3 Dashboard

- ☐ Modo oscuro por defecto y toggle claro/oscuro operativo.
- ☐ Filtros de Tiempo Real afectan solo vista izquierda.
- ☐ Filtros de Red Logistica afectan solo vista derecha.
- ☐ `TODOS -> TODOS` funciona en ambos contextos.
- ☐ No aparecen rutas espurias tipo `X -> X` en vehiculos.
- ☐ Panel de vehiculo muestra ruta, rumbo, siguiente nodo y ETA consistentes.
- ☐ Etiquetas de nodo RT en formato 3 letras y semitransparentes.
- ☐ Selectores origen/destino ordenados alfabeticamente.
- ☐ Tablas del dashboard ordenables por columna.
- ☐ Perfiles de ruta disponibles: `balanced`, `fastest`, `resilient`, `eco`, `low_risk`, `reliable`.
- ☐ `Patron horario` y pesos `tiempo/riesgo/eco` afectan al calculo de ruta.
- ☐ `Evitar nodos` modifica candidatas de ruta.
- ☐ Boton `Reentrenar IA` operativo y estado visible.
- ☐ Endpoint `GET /api/ml/retrain/status` devuelve `state`, `advice`, `model_info` y `schedule_info`.

2.4.4 3.4 Documentacion

- ☐ Memoria tecnica y resumen ejecutivo actualizados.
- ☐ Dashboard/Operations/Release notes alineados con implementacion final.
- ☐ README consistente con comportamiento actual del sistema.

2.5 4. Comandos de validacion rapida

```
./scripts/start_kdd.sh
docker compose ps
curl -s http://localhost:8501/health
curl -s http://localhost:8501/api/debug/sources
./scripts/validate_hive_pipeline.sh
```

Nota:

- `start_kdd.sh` ya ejecuta validacion de compatibilidad Hive streaming y sanity check de vistas Madrid.

2.6 5. Generacion de PDFs de entrega

PDF unificado (texto plano consolidado):

```
python3 scripts/build_delivery_pdf.py
```

Salida esperada:

- `docs/entrega-unificada-YYYY-MM-DD.pdf` (ej. `docs/entrega-unificada-2026-04-03.pdf`)

PDF profesional (maquetado con `pandoc/latex`):

```
./scripts/build_delivery_pdf_professional.sh
```

Salida esperada:

- `docs/entrega-unificada-profesional-YYYY-MM-DD.pdf` (ej. `docs/entrega-unificada-profesional-2026`

2.7 6. Nota final

Si se realizan cambios de frontend antes de defensa/demo, usar recarga fuerte del navegador:

```
Ctrl+F5
```

3 Resumen Ejecutivo - Sistema KDD Logistico

3.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Resumen ejecutivo para memoria`
- Version: `v1.4`
- Fecha: `04/04/2026`
- Repositorio GitHub: `https://github.com/raulsistemasydesarrollo/KDD-Ingenieria-de-datos`

3.2 Indice

1. Que se ha construido
2. Arquitectura y flujo de valor
3. Resultados funcionales alcanzados
4. Robustez y operaciones
5. Estado tecnico final (entrega)
6. Entregables documentales
7. Conclusiones

Este resumen sintetiza la solucion implementada para la entrega del proyecto, con foco en arquitectura, valor funcional, robustez operativa y estado final.

3.3 1. Que se ha construido

Se ha desarrollado una plataforma Big Data end-to-end para analisis logistico que integra:

1. Ingesta de eventos GPS y meteorologia en tiempo casi real.
2. Procesamiento batch y streaming sobre Spark.
3. Analitica de red logistica con grafos y rutas optimizadas.
4. Scoring de riesgo de retraso mediante ML.
5. Persistencia en HDFS/Hive y estado operativo en Cassandra.
6. Orquestacion y monitorizacion con Airflow.
7. Dashboard web operativo para explotacion funcional.

El sistema esta empaquetado en Docker Compose, con scripts de arranque, validacion y limpieza para reproducibilidad local.

3.4 2. Arquitectura y flujo de valor

3.4.1 Ingesta

- NiFi organiza la ingesta en dos subflujos claramente separados:
  - `gps_ingestion`
  - `weather_ingestion`
- Publica en Kafka cuatro topics principales:
  - `transport.raw`
  - `transport.filtered`
  - `transport.weather.raw`
  - `transport.weather.filtered`
- Conserva raw local y un proceso dedicado (`raw-hdfs-loader`) lo sincroniza a HDFS para trazabilidad y reprocesado.

3.4.2 Procesamiento

- Spark batch construye tablas historicas y analiticas:
  - enriquecimiento de eventos,
  - agregados de delay/velocidad,
  - metricas de grafo (`connected components`, `pagerank`),
  - shortest paths,
  - scoring ML de riesgo.
- Spark streaming procesa GPS y clima filtrado desde Kafka, con watermarks y deduplicacion, y persiste resultados en Hive.
- Spark streaming persiste ademas eventos enriquecidos en tiempo real en `transport_analytics.enriched_ever`

3.4.3 Almacenamiento y consulta

- Hive centraliza el catalogo analitico (batch y streaming).
- Cassandra mantiene el ultimo estado de cada vehiculo para consulta rapida.
- Cassandra almacena snapshots de insights de red para historico.
- HDFS mantiene capa raw y curated.

3.4.4 Explotacion

- Dashboard con:
  - mapa operativo de flota en tiempo real,
  - analisis de red logistica y mejor ruta por perfil,
  - impacto meteorologico en ETA y penalizacion de ruta.
- El dashboard usa Cassandra como fuente principal de flota y fallback a archivos NiFi si Cassandra no esta disponible.

- El dashboard usa plan de trayecto por vehiculo (`planned_origin`, `planned_destination`, `planned_route_nodes`) para reforzar coherencia de rutas visualizadas.
- Los filtros de Tiempo Real y de Red Logistica estan desacoplados para evitar interferencias entre vistas.

3.5 3. Resultados funcionales alcanzados

1. Visualizacion de vehiculos con trazas y foco por seleccion.
2. Calculo de ETA por vehiculo con suavizado para evitar fluctuaciones bruscas, incluyendo `ETA nodos restantes` y `ETA destino final`.
3. Calculo de mejor ruta por perfiles (`balanced`, `fastest`, `resilient`, `eco`, `low_risk`, `reliable`).
4. Incorporacion real del factor meteorologico en estimaciones de tiempo.
5. Representacion tabular de rutas y metricas (distancia, tiempos, penalizacion clima, factor aplicado).
6. Modo oscuro por defecto y cambio claro/oscuro en frontend.
7. Filtros RT parciales y completos (`TODOS->X`, `X->TODOS`, `X->Y`) con limpieza de seleccion fuera de filtro.
8. Insights live de red (cuellos de botella + nodos criticos + historico).
9. Tablas ordenables y selectores alfabeticos en ambas vistas.
10. Optimizacion multiobjetivo en routing con pesos `tiempo/riesgo/eco`, modo temporal (`auto`, `peak`, `offpeak`, `night`) y exclusion de nodos intermedios.
11. Operativa de reentreno IA desde dashboard:

- trigger manual (`POST /api/ml/retrain`),
- estado/recomendacion (`GET /api/ml/retrain/status`),
- score de deriva y logica de recomendacion con histeresis + cooldown,
- paneles de cabecera con modelo en uso destacado y descripcion de candidatos.

3.6 4. Robustez y operaciones

Se incorporaron mejoras de estabilidad relevantes para entrega:

1. Healthchecks Airflow (diario y horario) con degradacion segura en consultas smoke, evitando falsos fallos cuando las tablas fuente streaming no estan disponibles temporalmente.
2. Limpieza controlada de historico fallido en Airflow para dejar estado operativo en verde.
3. Reorganizacion visual de NiFi por dominios (GPS/Clima), simplificando soporte y revisiones.
4. Script de limpieza de Process Groups legacy de NiFi para mantener entorno limpio.
5. Fallback Parquet en Spark streaming si falla escritura a Hive, evitando parada del pipeline.

3.7 5. Estado tecnico final (entrega)

- NiFi:
  - Process Group activo: `kdd_ingestion_auto_v9`
  - Subgrupos: `gps_ingestion`, `weather_ingestion`
- Airflow:

  - DAGs de healthcheck estabilizados y sin failed pendientes en la validacion operativa.
- Dashboard:
  - Endpoint de diagnostico de fuentes: `/api/debug/sources`
  - Fuente vehiculos: Cassandra (primaria), `nifi/input` (fallback).
  - Endpoint de historico insights: `/api/network/insights/history`.
  - Endpoints ML de operacion: `/api/ml/retrain` y `/api/ml/retrain/status`.
  - Red consolidada en 15 nodos y flota de 15 vehiculos (14 activos + 1 mantenimiento).

3.8 6. Entregables documentales

Se han dejado documentos para operacion y defensa:

1. Memoria tecnica completa:
   - `docs/memoria-tecnica-sistema.md`
2. Operaciones y checklist:
   - `docs/operations.md`
3. Dashboard y leyenda funcional:
   - `docs/dashboard.md`
4. Release notes de iteracion:
   - `docs/release-notes-2026-04-04.md`

3.9 7. Conclusiones

La solucion cumple los objetivos del proyecto KDD en un entorno reproducible y demostrable:

- separa correctamente ingesta, transformacion, analitica y visualizacion,
- combina batch + streaming con gobernanza de datos,
- incorpora analitica avanzada (grafos y ML),
- ofrece mecanismos de observabilidad, resiliencia y mantenimiento.

Como resultado, se dispone de un sistema funcional para analisis logistico en tiempo real y soporte de decisiones operativas.

4 Memoria Tecnica del Sistema KDD Logistico

4.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Memoria tecnica del sistema`
- Version: `v1.4`
- Fecha: `04/04/2026`
- Repositorio GitHub: `https://github.com/raulsistemasydesarrollo/KDD-Ingenieria-de-datos`

4.2 Indice

1. Objetivo del sistema

2. Arquitectura general
3. Flujo de datos end-to-end
4. NiFi en detalle
5. Generacion y sincronizacion raw
6. Spark en detalle
7. Hive: tablas y vistas
8. Cassandra
9. Airflow en detalle
10. Dashboard
11. Orquestacion de arranque y reset
12. Operacion y validacion recomendada
13. Puntos de resiliencia implementados
14. Limitaciones conocidas

Documento de entrega con explicacion detallada del funcionamiento extremo a extremo del sistema: ingesta, procesamiento batch/streaming, almacenamiento, orquestacion y visualizacion.

Resumen ejecutivo asociado (1-2 paginas):

- `docs/resumen-ejecutivo-memoria.md`

4.3 1. Objetivo del sistema

El sistema implementa un ciclo KDD para analitica logistica, combinando:

- ingesta continua de GPS y meteorologia,
- procesamiento batch y streaming con Spark,
- analitica de red con grafos (GraphFrames),
- scoring de riesgo de retraso con ML,
- persistencia en Hive/HDFS y Cassandra,
- monitorizacion/orquestacion con Airflow,
- visualizacion web en dashboard.

4.4 2. Arquitectura general

Componentes principales del `docker-compose`:

1. `kafka` (Apache Kafka 3.9.1): bus de eventos.
2. `nifi` (Apache NiFi 2.7.0): ingesta y enrutamiento.
3. `gps-generator`: generador de eventos GPS simulados.
4. `hadoop`: HDFS + YARN (nodo unico pseudo-distribuido).
5. `raw-hdfs-loader`: sincroniza raw de NiFi hacia HDFS.
6. `hive-metastore` + `hive-server`: catalogo y consulta SQL.
7. `spark-client`: ejecucion de jobs Spark (batch y streaming).
8. `cassandra`: estado mas reciente por vehiculo para baja latencia.
9. `airflow-webserver` + `airflow-scheduler` + `airflow-postgres`: orquestacion.
10. `dashboard`: API HTTP + frontend para operacion y analitica visual.

4.4.1 Diagrama de arquitectura

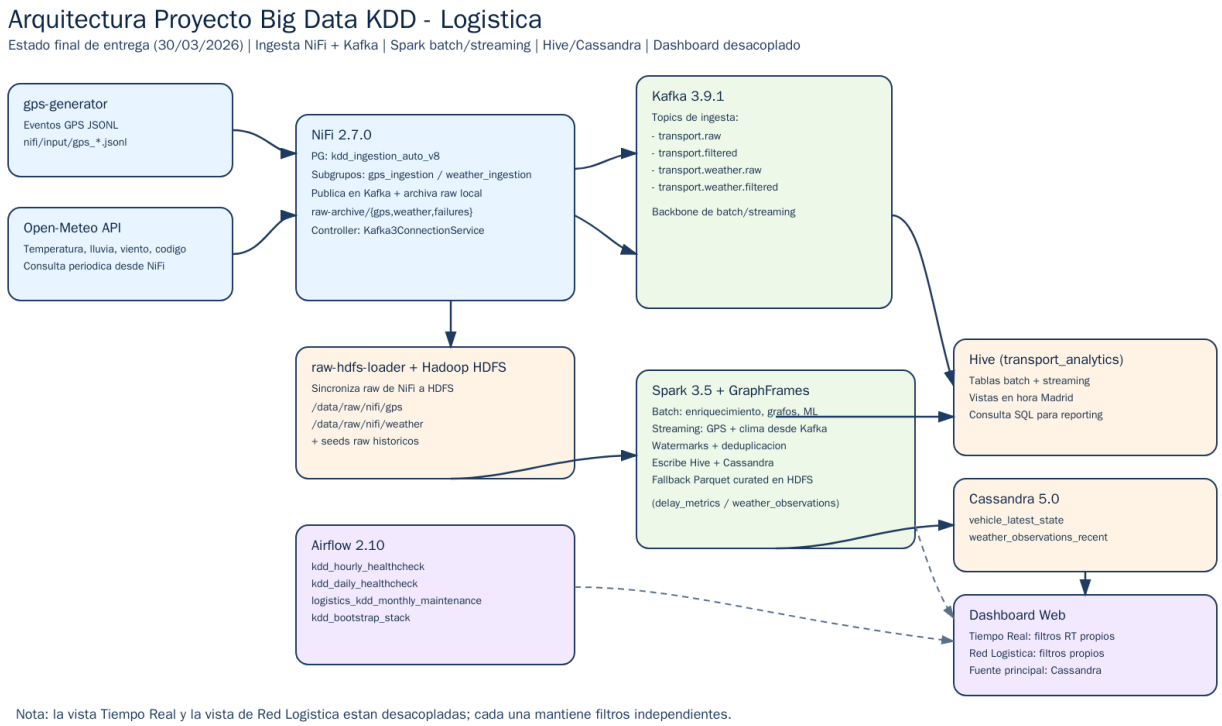

Figure 1: Diagrama de arquitectura del proyecto

Archivo fuente del diagrama:

- `docs/architecture-diagram.png`

4.5 3. Flujo de datos end-to-end

4.5.1 3.1 GPS

1. `gps-generator` crea ficheros `gps_*.jsonl` en `nifi/input`.
2. NiFi lee esos ficheros, extrae atributos y publica:
   - `transport.raw` (todos los eventos),
   - `transport.filtered` (eventos con `delay_minutes >= 5`).
3. NiFi archiva raw local en `nifi/raw-archive/gps`.
4. `raw-hdfs-loader` mueve esos raw a HDFS: `/data/raw/nifi/gps/...`.
5. Spark streaming consume `transport.filtered`, agrega ventanas y escribe:
   - `transport_analytics.delay_metrics_streaming` (Hive),
   - `transport_analytics.enriched_events_streaming` (Hive),
   - fallback Parquet: `/data/curated/delay_metrics_streaming`.

4.5.2 3.2 Meteorologia

1. NiFi consulta Open-Meteo periodicamente.
2. NiFi extrae y normaliza campos de clima.

3. NiFi publica:
   - `transport.weather.raw`,
   - `transport.weather.filtered`.
4. NiFi archiva raw local en `nifi/raw-archive/weather`.
5. `raw-hdfs-loader` sincroniza a HDFS: `/data/raw/nifi/weather/....`
6. Spark streaming consume `transport.weather.filtered` y escribe:
   - vista operativa de consulta: `transport_analytics.v_weather_observations_madrid`,
   - fallback Parquet: `/data/curated/weather_observations_streaming`.

4.5.3 3.3 Batch historico

1. Spark batch lee `hdfs://hadoop:9000/data/raw/gps_events.jsonl`.
2. Limpia/normaliza y enriquece con maestros de almacenes/vehiculos.
3. Calcula agregados, grafos y modelo ML.
4. Publica tablas analiticas en Hive.

4.6 4. NiFi en detalle

4.7 4.1 Estructura visual desplegada

Capturas de referencia de los subflujos:

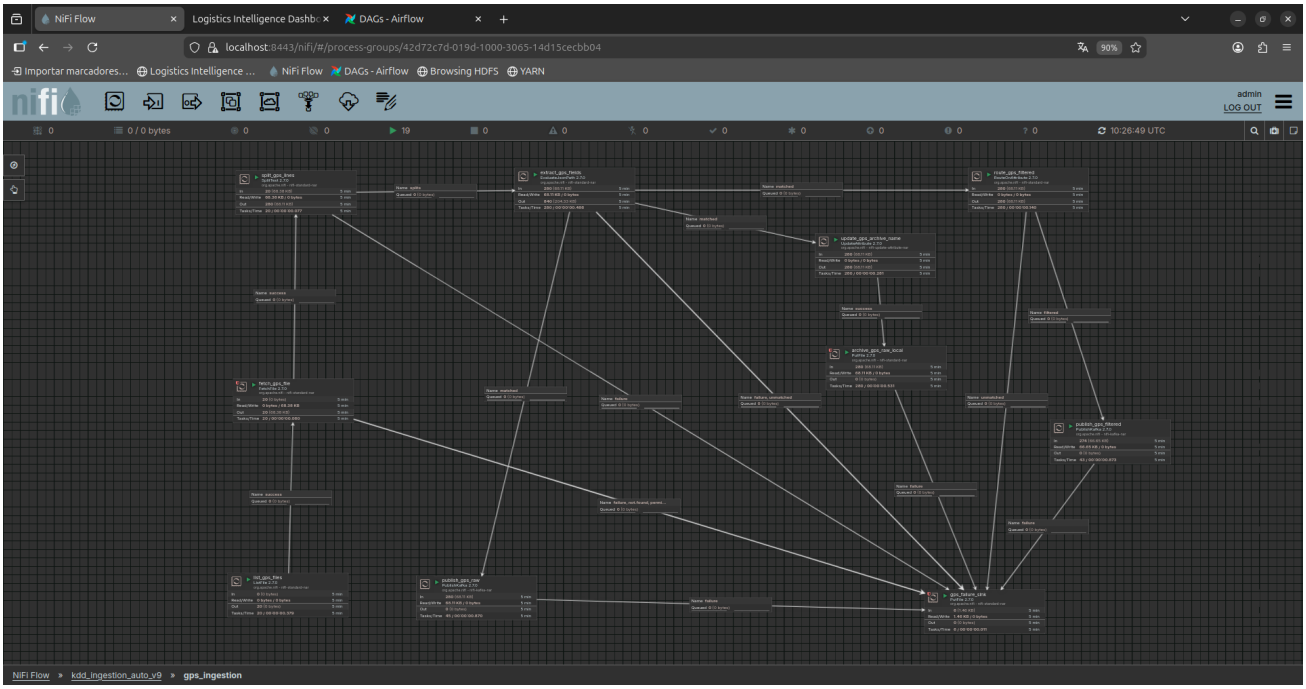

Figure 2: NiFi GPS ingestion

Bootstrap actual (`scripts/bootstrap_nifi_flow.sh`) crea:

- Process Group principal: `kdd_ingestion_auto_v9`
- Subgrupos:
  - `gps_ingestion`

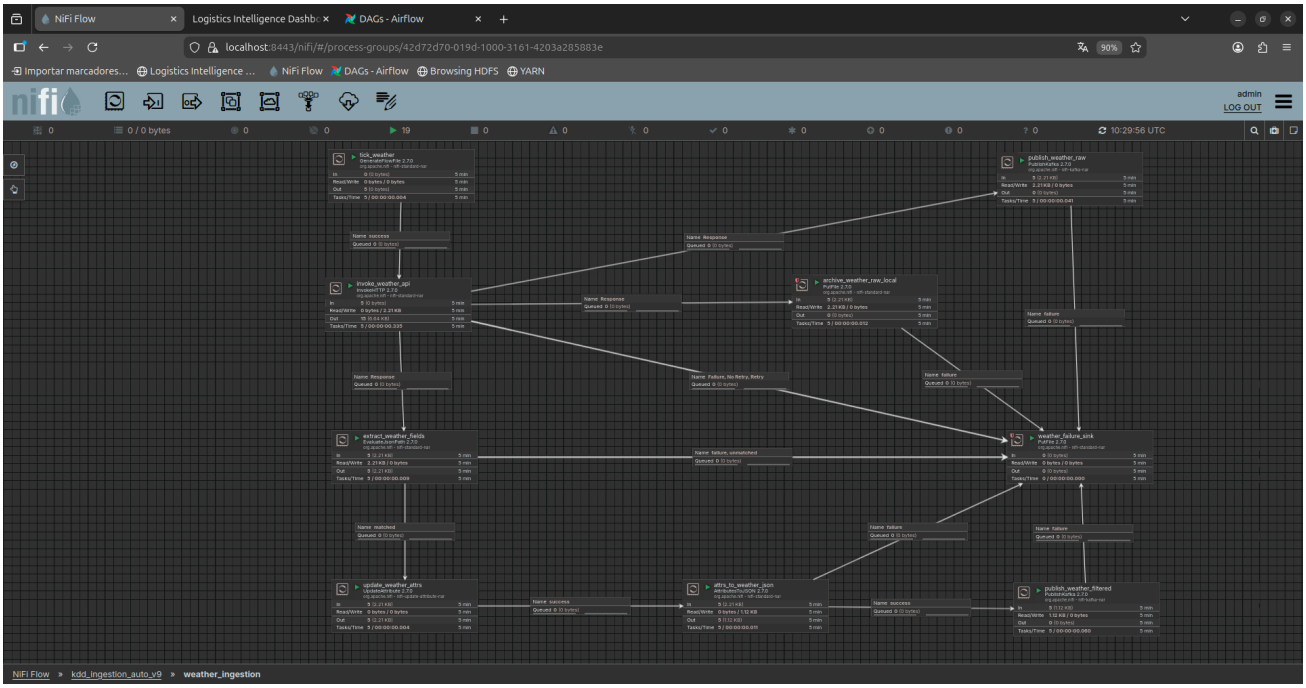

Figure 3: NiFi WEATHER ingestion

  - `weather_ingestion`

Esta separacion facilita mantenimiento y lectura en la UI.

4.8 4.2 Controller Service

- `kafka3_connection_service` (`Kafka3ConnectionService`)
  - `bootstrap.servers = kafka:9092`
  - compartido por procesadores `PublishKafka`.

4.9 4.3 Flujo GPS: procesadores y rol

1. `list_gps_files` (`ListFile`)
   - dir: `/opt/nifi/nifi-current/input`
   - filtro: `.*\.jsonl`
   - periodo: `15 sec`.
2. `fetch_gps_file` (`FetchFile`)
3. `split_gps_lines` (`SplitText`)
   - `Line Split Count=1`.
4. `extract_gps_fields` (`EvaluateJsonPath`)
   - extrae: `vehicle_id`, `warehouse_id`, `event_type`, `delay_minutes`.
5. `update_gps_archive_name` (`UpdateAttribute`)
   - `filename = ${filename}_${fragment.index}_${UUID()}.jsonl`.
6. `route_gps_filtered` (`RouteOnAttribute`)
   - regla `filtered`: `${delay_minutes:toNumber():ge(5)}`.

7. `publish_gps_raw` (`PublishKafka`)
    - topic: `transport.raw`.
8. `publish_gps_filtered` (`PublishKafka`)
    - topic: `transport.filtered`.
9. `archive_gps_raw_local` (`PutFile`)
    - dir: `/opt/nifi/nifi-current/raw-archive/gps`.
10. `gps_failure_sink` (`PutFile`)

- dir: `/opt/nifi/nifi-current/raw-archive/failures/gps`.

Ruteo principal:

- `matched` de `extract_gps_fields` pasa por `update_gps_archive_name` antes de archivado local (evita sobrescritura por split).
- `filtered` de `route_gps_filtered` va a Kafka filtrado.
- Errores y `unmatched` van a `gps_failure_sink`.

4.10 4.4 Flujo Clima: procesadores y rol

1. `tick_weather` (`GenerateFlowFile`)
    - periodo: `60 sec`.
2. `invoke_weather_api` (`InvokeHTTP`)
    - URL Open-Meteo (Madrid).
3. `extract_weather_fields` (`EvaluateJsonPath`)
    - `temperature_c`, `precipitation_mm`, `wind_kmh`, `weather_code`.
4. `update_weather_attrs` (`UpdateAttribute`)
    - `weather_event_id=${UUID()}`
    - `warehouse_id=WH1`
    - `source=open-meteo`
    - `observation_time` en UTC.
5. `attrs_to_weather_json` (`AttributesToJSON`)
    - transforma atributos a JSON final.
6. `publish_weather_raw` (`PublishKafka`)
    - topic: `transport.weather.raw`.
7. `publish_weather_filtered` (`PublishKafka`)
    - topic: `transport.weather.filtered`.
8. `archive_weather_raw_local` (`PutFile`)
    - dir: `/opt/nifi/nifi-current/raw-archive/weather`.
9. `weather_failure_sink` (`PutFile`)
    - dir: `/opt/nifi/nifi-current/raw-archive/failures/weather`.

Ruteo principal:

- `Response` de `InvokeHTTP` alimenta raw Kafka, archivo raw y extraccion.
- Camino normal termina en `publish_weather_filtered`.
- `Failure/Retry/No Retry` y fallos de transformacion/publicacion van a `weather_failure_sink`.

4.11 4.5 Back-pressure y terminacion

Cada conexion creada por bootstrap aplica:

- `backPressureObjectThreshold = 10000`
- `backPressureDataSizeThreshold = 1 GB`
- `flowFileExpiration = 0 sec`

Ademas, se auto-terminan relaciones no usadas para evitar bloqueos por conexiones obligatorias sin destino.

4.12 5. Generacion y sincronizacion raw

4.13 5.1 GPS generator

Script: `scripts/gps_generator.py`.

Caracteristicas:

- Frecuencia: cada `15s`.
- Factor temporal simulado: `SIM_TIME_FACTOR=6`.
- Flota cargada desde `data/master/vehicles.csv` (IDs actuales `TRUCK-*`, filtrando por `status` activo), 1 evento por vehiculo y ciclo.
- Catalogo de rutas realistas calculado desde el grafo logistico y reasignacion dinamica al cerrar tramos.
- Movimiento progresivo entre ciudades (sin salto directo idealizado).
- Estado persistido en `.vehicle_path_state.json` para continuidad de trayectorias.

Campos emitidos por evento:

- `event_id`, `vehicle_id`, `warehouse_id`, `route_id`, `event_type`,
- `latitude`, `longitude`, `delay_minutes`, `speed_kmh`, `event_time`.

4.14 5.2 Raw HDFS loader

Script: `docker/hadoop/scripts/load-nifi-raw-to-hdfs.sh`.

Funcion:

- Escanea raw local de NiFi (`/opt/nifi-raw/gps`, `/opt/nifi-raw/weather`) cada 30s.
- Sube ficheros a `/data/raw/nifi/{gps|weather}` en HDFS.
- Mueve ficheros procesados a `.processed` local para evitar reprocesado.

4.15 6. Spark en detalle

Clase principal: `com.proyectobigdata.LogisticsAnalyticsJob`.

Modos:

- `batch` (job historico + grafos + ML)
- `streaming` (consumo continuo de Kafka)

- `insights-sync` (consolidacion de snapshots de insights Cassandra -> Hive)

Lanzadores:

- `spark-app/run-batch.sh`
- `spark-app/run-streaming.sh`
- `spark-app/run-insights-sync.sh`

Paquetes Spark usados:

- `spark-sql-kafka-0-10_2.12:3.5.5`
- `graphframes-spark3_2.12:0.9.0-spark3.5`
- `spark-cassandra-connector_2.12:3.5.1`

4.16 6.1 Preparacion comun

Al iniciar:

1. Configura Spark SQL timezone (`SPARK_SQL_TIMEZONE`, por defecto `Europe/Madrid`).
2. Habilita soporte Hive (`enableHiveSupport`).
3. Crea DB Hive `transport_analytics` si no existe.
4. Carga maestros Hive:
   - `transport_analytics.master_warehouses`
   - `transport_analytics.master_vehicles`
5. Inicializa esquema Cassandra:
   - keyspace `transport`
   - tabla `transport.vehicle_latest_state`.

4.17 6.2 Pipeline batch

Entrada:

- `hdfs://hadoop:9000/data/raw/gps_events.jsonl`

Pasos:

1. Limpieza y normalizacion (`cleanAndNormalizeEvents`):
   - `event_time -> event_timestamp`,
   - cast de `delay_minutes`, `speed_kmh`,
   - filtros de nulos clave.
2. Deduplicacion por `event_id`.
3. Enriquecimiento con maestros (`warehouse` + `vehicle`).
4. Persistencia:
   - Parquet curated: `/data/curated/enriched_events`
   - Hive: `transport_analytics.enriched_events`.
5. Agregacion por ventana de 15 min y almacen:
   - Hive: `transport_analytics.delay_metrics_batch`.
6. Grafo (`GraphFrames`):

    - connected components + pagerank
    - Hive: `transport_analytics.route_graph_metrics`.
7. Shortest paths con landmarks de almacenes criticos:
    - Hive: `transport_analytics.route_shortest_paths`.
8. Estado mas reciente por vehiculo a Cassandra.
9. Entrenamiento/scoring ML de retraso:
    - Hive: `transport_analytics.ml_delay_risk_scores`
    - Modelo: `hdfs://hadoop:9000/models/delay_risk_rf`.

4.18 6.3 Pipeline streaming

Entradas Kafka:

- `transport.filtered` (GPS filtrado)
- `transport.weather.filtered` (meteo filtrada)

Pasos GPS streaming:

1. Parse JSON desde `value`.
2. Limpieza y normalizacion.
3. Watermark de `20 minutes`.
4. Deduplicacion por `event_id`.
5. Enriquecimiento con maestros.
6. Ventana de 15 min por `warehouse_id`.
7. Escritura `foreachBatch` a Hive:
    - `transport_analytics.delay_metrics_streaming`.
    - `transport_analytics.enriched_events_streaming`.
8. Escritura de latest state por vehiculo a Cassandra.

Pasos clima streaming:

1. Parse de campos meteo desde JSON.
2. Limpieza y normalizacion (`weather_timestamp`, casts, filtros).
3. Watermark `20 minutes`.
4. Deduplicacion por `weather_event_id`.
5. Escritura `foreachBatch` a Hive:
    - consumo operativo via vista `transport_analytics.v_weather_observations_madrid`.

Fallback robustez:

- Si falla escritura Hive (incompatibilidad/metastore), guarda en Parquet fallback:
    - `/data/curated/delay_metrics_streaming`
    - `/data/curated/enriched_events_streaming`
    - `/data/curated/weather_observations_streaming`

Checkpoints:

- `/tmp/checkpoints/delay_metrics`

- `/tmp/checkpoints/latest_vehicle_state`
- `/tmp/checkpoints/weather_observations`

Compatibilidad operativa:

- El script `scripts/ensure_hive_streaming_compat.sh` asegura que existan:
  - `transport_analytics.delay_metrics_streaming`
  - `transport_analytics.weather_observations_streaming`
  - vistas Madrid asociadas.
- Se ejecuta en arranque para evitar vistas "colgadas" tras limpiezas.

4.19 6.4 ML de riesgo de retraso

Modelo principal:

- `RandomForestRegressor`
- estrategia de seleccion:
  - `baseline_rf`: `speed_kmh` + (`warehouse_id`, `vehicle_type`) codificados,
  - `tuned_baseline_rf`: mismas features base con hiperparametros ampliados,
  - `enhanced_rf`: añade `route_id`, `vehicle_id`, `capacity_kg`, `vehicle_status_score`, `warehouse_criticality_score`, `distance_to_warehouse_km`, `hour_sin/cos`, `is_weekend`, y contexto por almacen de clima (`climate_temperature_c`, `climate_precipitation_mm`, `climate_wind_kmh`, `climate_severity_score`) y congestion (`congestion_avg_delay_minutes`, `congestion_avg_speed_kmh`, `congestion_event_count`, `congestion_pressure_score`).
  - El contexto de clima/congestion en `enhanced_rf` se alinea por ventana temporal de 15 minutos (`event_timestamp`).
- label: `delay_minutes`

Logica:

- si dataset >= 3 filas: entrena modelo (con split 80/20 si >=20).
- evalua RMSE en test para los 3 candidatos.
- selecciona automaticamente el modelo con menor RMSE y lo persiste en:
  - `hdfs://hadoop:9000/models/delay_risk_rf`.
- puntua ultimo estado por vehiculo y clasifica riesgo:
  - `high` (>=10)
  - `medium` (>=5)
  - `low` (<5)

Fallback heuristico (si dataset pequeno):

- regla por `speed_kmh` para `predicted_delay_minutes`.

Resultado validado en la iteracion `04/04/2026` (dataset semilla 20k):

- `baseline_rmse=6.1329`
- `tuned_baseline_rmse=6.1343`
- `enhanced_rmse=6.0206`

- modelo seleccionado: `enhanced_rf`.
- tamano de modelo en HDFS: ~`1.2 MB`.

4.20 7. Hive: tablas y vistas

4.21 7.1 Tablas maestras y batch

- `transport_analytics.master_warehouses`
- `transport_analytics.master_vehicles`
- `transport_analytics.enriched_events`
- `transport_analytics.delay_metrics_batch`
- `transport_analytics.route_graph_metrics`
- `transport_analytics.route_shortest_paths`
- `transport_analytics.ml_delay_risk_scores`

4.22 7.2 Tablas streaming

- `transport_analytics.delay_metrics_streaming`
- `transport_analytics.enriched_events_streaming`
- (opcional segun entorno) `transport_analytics.weather_observations_streaming`
- `transport_analytics.network_insights_snapshots_hive`
- `transport_analytics.network_insights_hourly_trends`

4.23 7.3 Vistas en hora Madrid

Vistas usadas por healthchecks/dashboard:

- `transport_analytics.v_weather_observations_madrid`
- `transport_analytics.v_delay_metrics_streaming_madrid`

Nota operativa:

- Si las tablas streaming fuente no estan disponibles temporalmente, los DAGs de healthcheck degradan a validaciones seguras para no producir falsos rojos.
- En esta entrega, para consultas meteo de operacion se usa `transport_analytics.v_weather_observations_m`

4.24 8. Cassandra

Keyspace/tablas:

- `transport.vehicle_latest_state`
- `transport.weather_observations_recent`
- `transport.network_insights_snapshots`

Uso:

- mantener ultimo estado por `vehicle_id` para consulta rapida.

Campos:

- `vehicle_id` (PK), `warehouse_id`, `route_id`,
- `last_event_timestamp`, `delay_minutes`, `speed_kmh`,
- `latitude`, `longitude`.

4.25 9. Airflow en detalle

Captura de referencia del entorno Airflow:

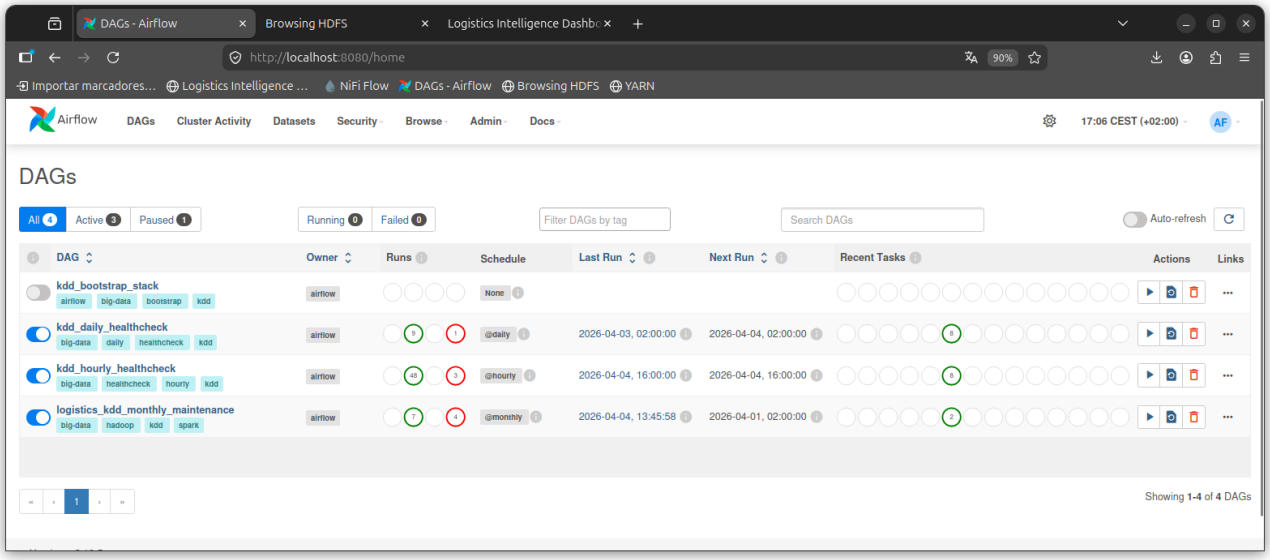

Figure 4: Airflow

DAGs:

1. `kdd_bootstrap_stack`
   - Manual.
   - Arranca servicios no-Airflow con `docker compose up`.
2. `kdd_hourly_healthcheck` (`@hourly`)
3. `kdd_daily_healthcheck` (`@daily`)
4. `logistics_kdd_monthly_maintenance` (`@monthly`)
   - ejecuta batch Spark y limpia checkpoints temporales.

Healthchecks (hourly/daily):

- contenedores core,
- topics Kafka obligatorios,
- rutas HDFS,
- tablas Hive base,
- vistas Madrid,
- smoke query sobre vistas (con fallback seguro).

Alertas:

- `on_failure_callback` en todos los DAGs,

- opcional email via `AIRFLOW_ALERT_EMAIL`.

4.26 10. Dashboard

Captura de estado operativo del dashboard:

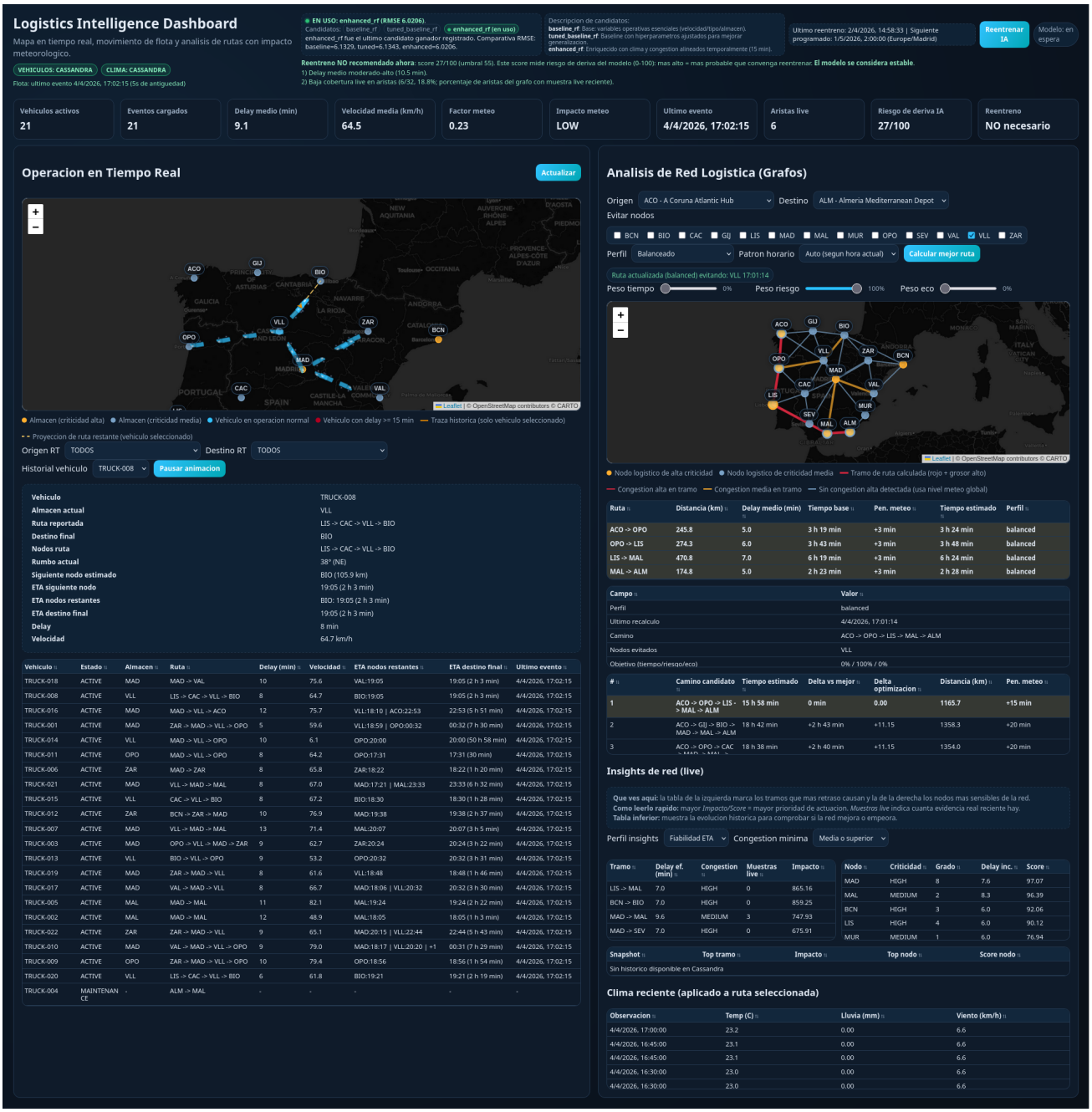

Figure 5: Dashboard logistica (captura actualizada 04/04/2026)

Backend:

- `dashboard/server.py` (HTTP server multihilo).
- Lee:

  - estado de vehiculo desde Cassandra (`transport.vehicle_latest_state`) como fuente primaria,
  - fallback de GPS desde `nifi/input` si Cassandra no esta disponible,
  - plan de trayecto por vehiculo desde `nifi/input/.vehicle_path_state.json` (`planned_origin`, `planned_destination`),
  - clima reciente desde Cassandra (`transport.weather_observations_recent`) como fuente primaria,
  - fallback de clima con snapshots en `nifi/raw-archive/weather`,
  - grafo y maestros desde `data/graph/*.csv`, `data/master/warehouses.csv`.

API principal:

- `/api/overview`
- `/api/vehicles/latest`
- `/api/vehicles/history`
- `/api/weather/latest`
- `/api/network/graph`
- `/api/network/best-route`
- `/api/network/insights/history`
- `/api/ml/retrain` (POST, trigger de reentreno)
- `/api/ml/retrain/status` (GET, estado + recomendacion)
- `/api/debug/sources` (diagnostico de fuentes activas y fallback)

Motor de ruta:

- Dijkstra en backend con perfiles:
  - `balanced`, `fastest`, `resilient`, `eco`, `low_risk`, `reliable`.
- Peso de arista considera distancia, delay efectivo, congestion, meteo, incertidumbre y pesos multiobjetivo (`time/risk/eco`).
- Soporta exclusiones de nodos (`avoid_nodes`) y modo temporal (`auto`, `peak`, `offpeak`, `night`).
- Devuelve explicabilidad de ruta (`explain`) y metricas de fiabilidad (`on_time_probability`).

Frontend:

- modo oscuro por defecto (toggle claro/oscuro),
- mapa tiempo real y mapa de red logistica,
- tabla de rutas con ETA y desglose de tiempos.
- tablas ordenables por columna.
- filtros desacoplados por vista:
  - Tiempo Real: `Origen RT` / `Destino RT` (solo columna izquierda),
  - Red Logistica: `Origen` / `Destino` / `Perfil` (solo columna derecha).
- selector de `Perfil` ampliado a 6 estrategias (`balanced`, `fastest`, `resilient`, `eco`, `low_risk`, `reliable`).
- controles de optimizacion multiobjetivo:
  - `Peso tiempo`,
  - `Peso riesgo`,

  - `Peso eco`.
- selector de `Patron horario` (`auto`, `peak`, `offpeak`, `night`) y panel `Evitar nodos`.
- bloque de reentreno IA en cabecera:
  - boton `Reentrenar IA`,
  - estado runtime,
  - recomendacion de reentreno con score de deriva y motivos,
  - panel de modelos IA en dos bloques:
    * izquierda: `EN USO` + candidato elegido (`A/B/C`) y comparativa RMSE,
    * derecha: descripcion funcional de los 3 candidatos.
- comportamiento de filtros RT:
  - `TODOS->TODOS` vista global,
  - filtros parciales (`TODOS->X` o `X->TODOS`) soportados.
- reglas de consistencia:
  - no ruta concreta con origen=destino en bloque logistico,
  - limpieza automatica de seleccion de vehiculo si deja de cumplir filtro RT,
  - priorizacion de `planned_origin/planned_destination` + `planned_route_nodes` para evitar rutas `X->X` espurias y proyecciones incoherentes.
  - uso de `planned_route_nodes` para proyectar ruta restante (no nodos ya superados).
  - selectores de origen/destino ordenados alfabeticamente en ambas vistas,
  - etiquetas de nodo RT en formato 3 letras y semitransparentes para no ocultar vehiculos.

4.27 11. Orquestacion de arranque y reset

Script principal de arranque:

- `scripts/start_kdd.sh`
  - levanta stack,
  - espera NiFi,
  - ejecuta bootstrap NiFi (por defecto `kdd_ingestion_auto_v9`),
  - asegura compatibilidad Hive streaming,
  - repuebla meteo Hive si procede,
  - verifica dashboard.

Reset de demo:

- `scripts/reset_demo_data.sh`
  - limpia eventos locales,
  - resetea estado streaming (Hive + checkpoints),
  - recrea generadores.

4.28 12. Operacion y validacion recomendada

Arranque:

```
./scripts/start_kdd.sh
```

Validacion E2E batch+streaming:

```
./scripts/validate_hive_pipeline.sh
```

Estado de reentreno IA (dashboard):

```
curl -s http://localhost:8501/api/ml/retrain/status
```

Trigger manual de reentreno IA:

```
curl -s -X POST http://localhost:8501/api/ml/retrain \
  -H 'Content-Type: application/json' \
  -d '{"trigger":"manual_dashboard"}'
```

Limpieza runs rojos historicos (healthchecks):

```
./scripts/cleanup_airflow_failed_runs.sh --apply
```

Limpieza de Process Groups legacy NiFi:

```
./scripts/cleanup_nifi_legacy_pgs.py
```

4.29 13. Puntos de resiliencia implementados

1. Fallback a Parquet cuando Hive streaming falla.
2. Manejo de excepciones en escritura Cassandra (no bloquea pipeline).
3. Healthchecks Airflow con degradacion segura en smoke queries.
4. Colas de error separadas en NiFi (`failure sinks` GPS/Clima).
5. Separacion visual de flujos NiFi para diagnostico rapido.
6. Reentreno IA no bloqueante en dashboard (worker asincrono + timeout configurable).
7. Recomendacion de reentreno con histeresis y cooldown para evitar ejecuciones innecesarias.

4.30 14. Limitaciones conocidas

1. El dashboard depende de Cassandra para `/api/vehicles/latest`; si Cassandra cae, usa fallback desde `nifi/input` y puede perderse parte del estado reciente.
2. Si Cassandra no esta disponible, la precision de clima en dashboard pasa a depender del fallback de snapshots en `raw-archive/weather`.
3. En entorno local, tiempos de arranque de NiFi/Spark pueden variar segun recursos.

Esta memoria describe el estado tecnico actual del proyecto para entrega academica y operacion reproducible del entorno.

5 Flujo NiFi sugerido

5.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Flujo NiFi operativo`
- Version: `v1.4`

- Fecha: `04/04/2026`
- Repositorio GitHub: `https://github.com/raulsistemasydesarrollo/KDD-Ingenieria-de-datos`

5.2 Objetivo

Crear una ingesta dual hacia Kafka y HDFS:

- tema `transport.raw`
- tema `transport.filtered`
- tema `transport.weather.raw`
- tema `transport.weather.filtered`
- copia raw en HDFS

5.3 Flujo minimo (GPS)

Captura del subflujo desplegado en NiFi:

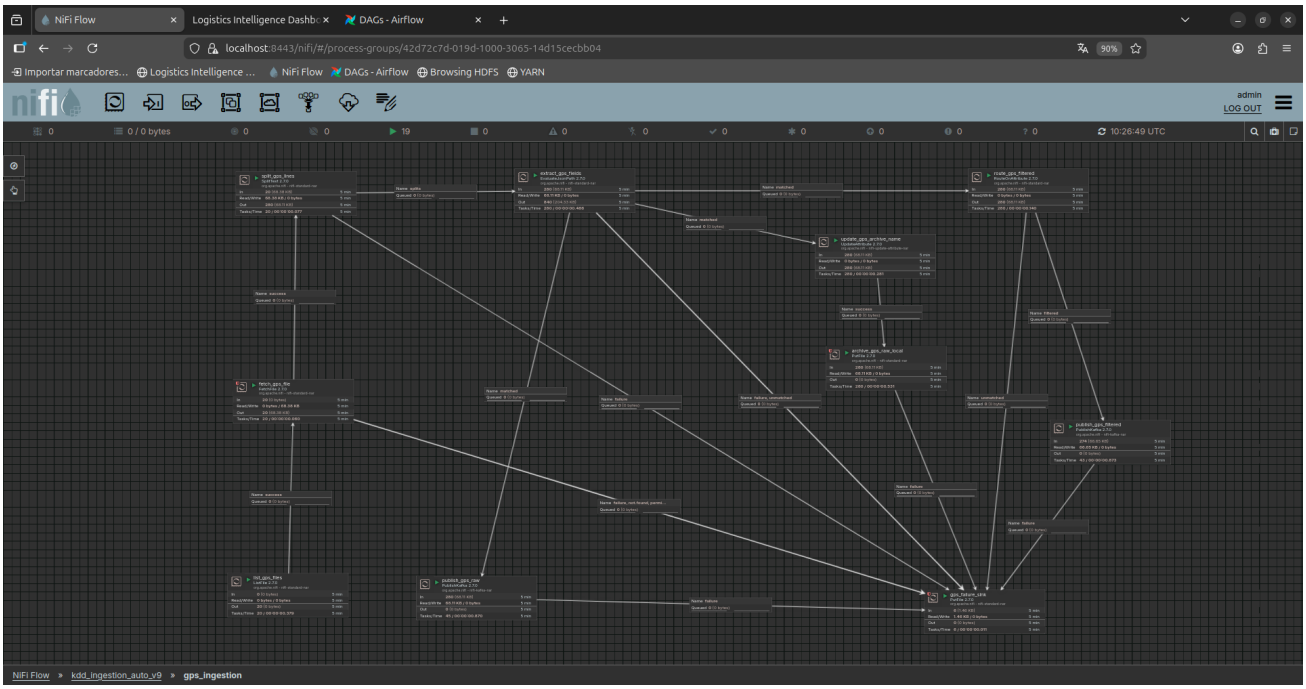

Figure 6: NiFi GPS ingestion

1. `ListFile`
   - Directorio: `/opt/nifi/nifi-current/input`
   - Filtro: `*.jsonl`
2. `FetchFile`
3. `SplitText`
   - 1 linea por evento
4. `EvaluateJsonPath`
   - extraer `vehicle_id`, `warehouse_id`, `event_type`, `delay_minutes`
5. `UpdateAttribute`

- `filename = ${filename}_${fragment.index}_${UUID()}.jsonl`
- objetivo: evitar sobrescritura en `raw-archive/gps` cuando `SplitText` genera multiples flowfiles desde un mismo archivo.

6. `RouteOnAttribute`
   - `filtered: ${delay_minutes:toNumber():ge(5)}`
7. `PublishKafka` (NiFi 2.7 con `Kafka3ConnectionService`)
   - Topic raw (todos los GPS): `transport.raw`
8. `PublishKafka` (NiFi 2.7 con `Kafka3ConnectionService`)
   - Topic filtered: `transport.filtered`
9. `PutFile`
   - Directorio: `/opt/nifi/nifi-current/raw-archive/gps`
10. `raw-hdfs-loader`

- Sincroniza `nifi/raw-archive` -> HDFS `/data/raw/nifi`

5.4 Recomendaciones

- Habilitar `back pressure` en las conexiones NiFi.
- Configurar relaciones `failure` hacia una cola de reintentos.
- En el bootstrap actual se enrutan errores a:
  - `/opt/nifi/nifi-current/raw-archive/failures/gps`
  - `/opt/nifi/nifi-current/raw-archive/failures/weather`

5.5 Ingesta API meteorologica (requisito del enunciado)

Captura del subflujo de meteorologia desplegado en NiFi:

Flujo adicional recomendado para clima:

1. `GenerateFlowFile`
   - Scheduling: cada 60s
2. `InvokeHTTP`
   - Metodo: `GET`
   - URL API publica (Open-Meteo):
   - `https://api.open-meteo.com/v1/forecast?latitude=40.4168&longitude=-3.7038&current=tempe`
3. `EvaluateJsonPath`
   - Extraer `temperature_2m`, `precipitation`, `wind_speed_10m`
4. `AttributesToJSON` / `JoltTransformJSON`
   - Normalizar al contrato:
   - `weather_event_id`, `warehouse_id`, `temperature_c`, `precipitation_mm`, `wind_kmh`, `weather_code`, `source`, `observation_time`
5. `PublishKafka` (desde `InvokeHTTP` response)
   - Topic raw: `transport.weather.raw`
6. `PublishKafka` (desde JSON normalizado)
   - Topic filtrado: `transport.weather.filtered`

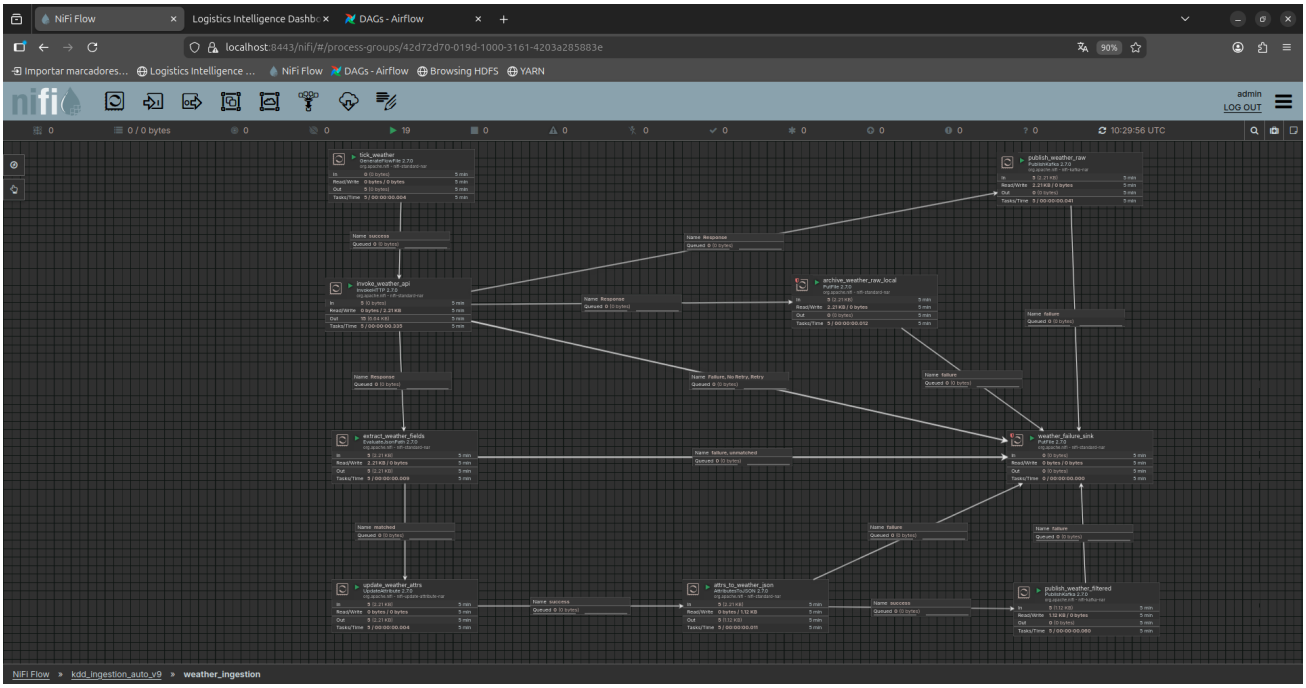

Figure 7: NiFi WEATHER ingestion

7. `PutFile`
   - Directorio raw local: `/opt/nifi/nifi-current/raw-archive/weather`
8. `raw-hdfs-loader`
   - Sincroniza a HDFS en `/data/raw/nifi/weather`

5.6 Generador GPS para NiFi

El proyecto incluye un generador continuo de logs GPS para alimentar `ListFile/FetchFile`:

- Servicio Docker: `gps-generator`
- Script: `scripts/gps_generator.py`
- Salida: `nifi/input/*.jsonl`

5.7 Bootstrap automatico NiFi

Sin configurar a mano la UI, puedes crear y arrancar el flujo con:

```
./scripts/bootstrap_nifi_flow.sh
```

Este bootstrap crea el Process Group indicado por `NIFI_PG_NAME` (por defecto `kdd_ingestion_auto_v9`) con:

- GPS simulado -> `transport.raw` y `transport.filtered`
- Meteo API publica -> `transport.weather.raw` y `transport.weather.filtered`
- Archivado raw local en `nifi/raw-archive/`

La subida continua a HDFS (`/data/raw/nifi`) la hace el servicio `raw-hdfs-loader`.

6 Dashboard Logistico

6.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Especificacion funcional del dashboard`
- Version: `v1.4`
- Fecha: `04/04/2026`
- Repositorio GitHub: `https://github.com/raulsistemasydesarrollo/KDD-Ingenieria-de-datos`

6.2 Indice

1. Objetivo
2. Fuentes de datos del dashboard
3. Que representa cada elemento visual
4. Filtros de Tiempo Real (solo vista izquierda)
5. Filtros de Analisis Logistico (solo vista derecha)
6. Reentreno IA (estado de modelo)
7. Reglas de interpretacion de ruta por vehiculo
8. ETA en tabla e historial de vehiculo
9. Datos y nodos actuales
10. Reinicio limpio de demo
11. Nota de uso

6.3 Objetivo

El dashboard muestra dos vistas operativas **desacopladas** (independientes), mas el bloque de metricas globales:

- Operacion en tiempo real de vehiculos (mapa y paneles de flota).
- Analisis de red logistica (mapa de red y tablas de rutas/impacto).

Manual de usuario completo del sistema (incluye dashboard + Airflow + operacion):

- `docs/manual-usuario.md`

6.4 Captura actual

6.5 Fuentes de datos del dashboard

- Vehiculos en tiempo real: fuente primaria `Cassandra` (`transport.vehicle_latest_state`).
- Fallback de vehiculos: ficheros `nifi/input/gps_*.jsonl` si Cassandra no esta disponible.
- Plan de ruta por vehiculo (origen/destino planificados): `nifi/input/.vehicle_path_state.json`.
- Clima reciente: fuente primaria `Cassandra` (`transport.weather_observations_recent`), cargada por Spark Streaming.
- Fallback de clima: `nifi/raw-archive/weather` (snapshots Open-Meteo) si Cassandra no esta disponible.

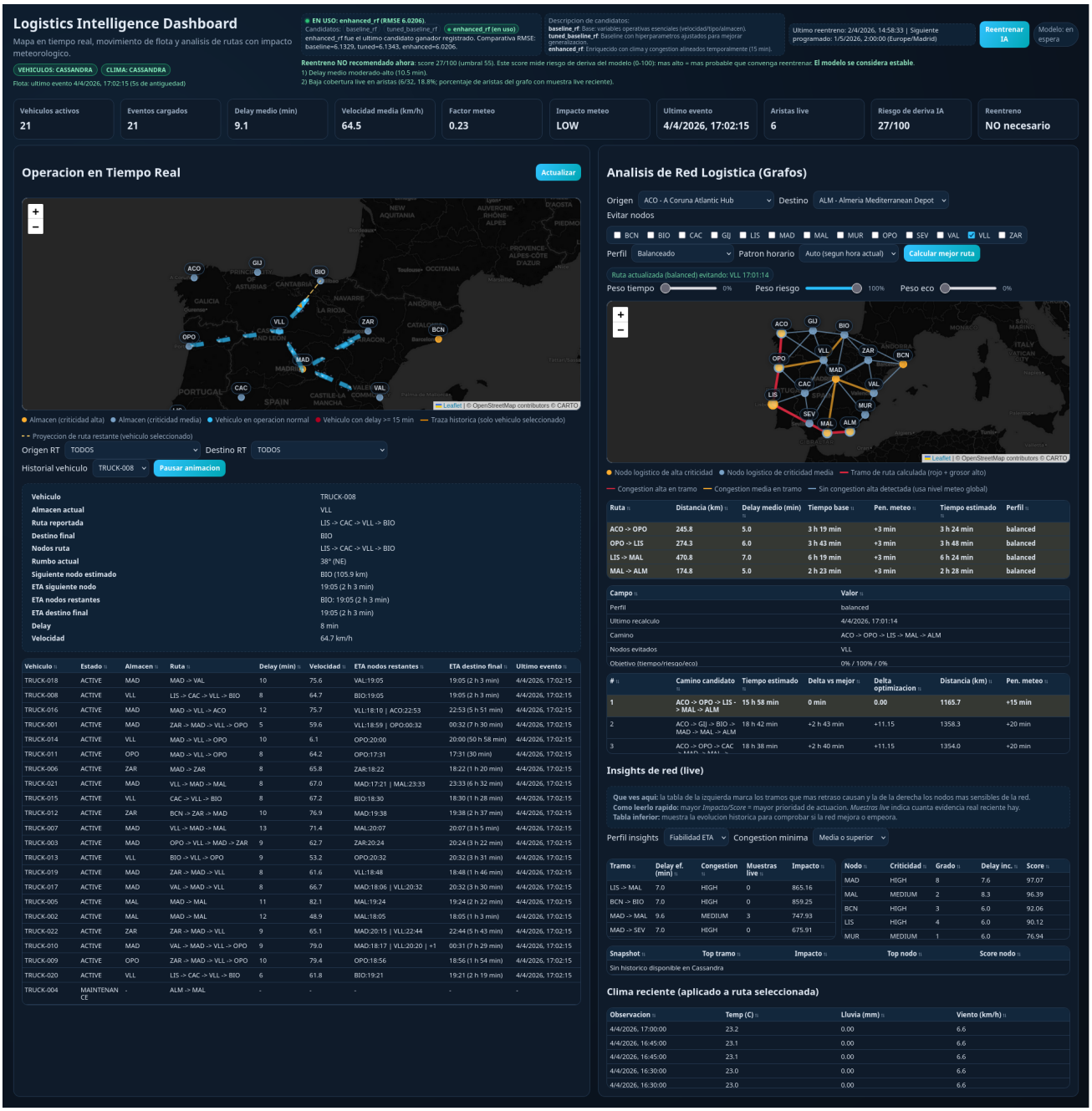

Figure 8: Dashboard logistica (captura actualizada 04/04/2026)

- Red logistica: `data/graph/vertices.csv`, `data/graph/edges.csv`, `data/master/warehouses.csv`.

Endpoint de diagnostico de fuentes:

```
curl -s http://localhost:8501/api/debug/sources
```

Devuelve:

- fuente activa para vehiculos (`cassandra` o `nifi_files`),
- estado de conexion a Cassandra y numero de filas obtenidas,
- ultimo fichero GPS detectado en fallback,
- fuente activa de clima (`cassandra` o `nifi_raw_archive`) y estado Cassandra de clima,
- estado/ficheros recientes de clima en fallback,
- conteo de vertices/aristas del grafo.

6.6 Endpoints operativos del dashboard

- `GET /api/overview`
- `GET /api/vehicles/latest`
- `GET /api/vehicles/history`
- `GET /api/weather/latest`
- `GET /api/network/graph`
- `GET /api/network/insights/history`
- `GET /api/network/best-route`
- `GET /api/debug/sources`
- `POST /api/ml/retrain`
- `GET /api/ml/retrain/status`

Parametros clave en `GET /api/network/best-route`:

- `source`, `target`
- `profile`
- `alternatives`
- `avoid_nodes` (CSV)
- `temporal_mode` (`auto`, `peak`, `offpeak`, `night`)
- `objective_time`
- `objective_risk`
- `objective_eco`

6.7 Que representa cada elemento visual

6.7.1 Mapa en tiempo real

- Marcadores de vehiculo: posicion actual por `vehicle_id`.
- Lineas naranjas: traza reciente del movimiento del vehiculo (historial corto), **no** son carreteras fijas.
- Linea discontinua amarilla: proyeccion de la ruta restante del vehiculo seleccionado (desde posicion actual hasta nodos pendientes).

- Marcadores de almacen:
  - naranja: criticidad `high`
  - azul: criticidad `medium`
  - etiqueta semitransparente para no ocultar vehiculos.
- Etiquetas de nodo mostradas en codigo de 3 letras (ej. `MAD`, `BCN`, `GIJ`), alineadas con la vista de red.

6.7.2 Filtros de Tiempo Real (solo vista izquierda)

- `Origen RT` y `Destino RT` filtran **solo** la vista de Operacion en Tiempo Real.
- Listado de `Origen RT` / `Destino RT` ordenado alfabeticamente.
- `TODOS -> TODOS`: sin filtro (vista global de flota).
- `TODOS -> X`: muestra vehiculos cuya ruta reportada termina en `X`.
- `X -> TODOS`: muestra vehiculos cuya ruta reportada comienza en `X`.
- `X -> Y`: restringe al corredor X-Y con criterio geografico y de ruta.
- Si el vehiculo seleccionado deja de cumplir el filtro, se limpia automaticamente la seleccion.

6.7.3 Mapa de red logistica

- Nodos: almacenes con coordenadas reales (`data/master/warehouses.csv`).
- Aristas: conexiones del grafo (`data/graph/edges.csv`).
- Arista roja: tramo perteneciente a la ruta calculada para el origen/destino seleccionados.
- Color de arista base: severidad meteorologica agregada (low/medium/high).
- Selectores `Origen` / `Destino` ordenados alfabeticamente.
- Delay de arista en tabla/mapa: valor **efectivo** (`effective_avg_delay_minutes`) que mezcla:
  - delay estatico del grafo (`avg_delay_minutes`),
  - telemetria viva de flota (`planned_origin/planned_destination`, delay y velocidad recientes).
- Cuando hay telemetria live en un tramo, se muestra etiqueta `live` en la tabla de rutas.

6.7.4 Tabla de ruta (debajo del mapa logistico)

Muestra:

- Perfil de optimizacion.
- Camino calculado.
- Distancia total.
- Tiempo base sin meteo.
- Penalizacion meteo (en minutos).
- Delay esperado.
- Tiempo estimado final.
- Factor meteorologico aplicado.
- Numero de aristas de la ruta con telemetria live usada en el calculo.
- Nota: el contador `Aristas live` refleja solo tramos de la **ruta seleccionada** con muestras live (`live_sample_count > 0`), no el total del grafo.

- Todas las tablas del dashboard son ordenables por columna (asc/desc) con click en cabecera.
- La tabla de resumen de ruta incorpora ademas:
  - `Objetivo (tiempo/riesgo/eco)` normalizado desde los sliders,
  - `Patron horario aplicado` (`auto`, `peak`, `offpeak`, `night`),
  - `Score riesgo`, `Score eco`, `Incertidumbre media`,
  - `Prob. llegada en hora` y bloque `Explicacion IA`.

6.7.5 Insights de red (live)

Bloque adicional en la vista de red con dos tablas:

- `Cuellos de botella`: ranking de tramos por `impact_score` (delay efectivo, congestion, distancia y muestras live).
- `Nodos criticos`: ranking de almacenes por `criticality_score` (grado, delay incidente medio, ratio de congestion alta y cobertura live).

Filtros disponibles:

- `Perfil insights`: recalcula score de impacto con los costes del perfil (`balanced`, `fastest`, `resilient`, `eco`, `low_risk`, `reliable`).
- `Congestion minima`: filtra el ranking para mostrar solo tramos/nodos afectados por el nivel indicado (`all`, `low`, `medium`, `high`).
- `Historico insights`: tabla de snapshots recientes persistidos en Cassandra por perfil/congestion.
- Si un tramo llega sin muestras live (`live_sample_count=0`) y congestion `UNKNOWN`, se normaliza a `low/medium/high` con heuristica de delay/distancia para mantener consistencia visual.

Endpoint historico:

```
curl -s "http://localhost:8501/api/network/insights/history?insights_profile=balanced&insights_m
```

6.8 Consolidacion en Hive (reporting)

Se incorpora modo Spark `insights-sync` para consolidar snapshots de Cassandra en Hive:

```
sg docker -c "docker compose exec -T spark-client /opt/spark-app/run-insights-sync.sh"
```

Tablas resultado:

- `transport_analytics.network_insights_snapshots_hive` (detalle de snapshots).
- `transport_analytics.network_insights_hourly_trends` (top tramo y top nodo por hora/perfil/congestion).

6.9 Grafo por cercania geografica (k vecinos)

Para regenerar `data/graph/edges.csv` con una red "casi completa" por cercania:

```
python3 scripts/rebuild_graph_edges_by_proximity.py --k 4
```

Alternativa con variable de entorno:

```
GRAPH_K_NEIGHBORS=5 python3 scripts/rebuild_graph_edges_by_proximity.py
```

Reglas de negocio que aplica el script al regenerar aristas:

- Conexiones directas prohibidas: `BCN-MUR`, `ACO-BIO`, `VAL-ALM`.
- Conexiones obligatorias: `BCN-VAL`, `VAL-MUR`, `ACO-GIJ`, `GIJ-BIO`, `MAD-SEV`, `MAD-MAL`.
- Corredor sur sin atajos directos: `CAC-SEV-MAL-ALM-MUR`.

6.9.1 Filtros de Analisis Logistico (solo vista derecha)

- `Origen` y `Destino` afectan **solo** al bloque de Analisis de Red Logistica.
- `TODOS` en alguno de los selectores pone el bloque en modo vista global (sin ruta concreta).
- No se permite ruta concreta con mismo origen y destino.
- `Calcular mejor ruta` aplica el perfil seleccionado solo al bloque logistico.
- Perfiles de optimizacion disponibles:
  - `balanced` (equilibrio general),
  - `fastest` (prioriza tiempo),
  - `resilient` (penaliza clima/congestion),
  - `eco` (menor coste consumo/emisiones),
  - `low_risk` (ETA estable, baja exposicion operativa),
  - `reliable` (maximiza fiabilidad de ETA).
- `Evitar nodos`: excluye nodos intermedios concretos del calculo.
- `Patron horario`: aplica factor temporal por franja (`auto`, `peak`, `offpeak`, `night`).
- Pesos de objetivo:
  - `Peso tiempo`
  - `Peso riesgo`
  - `Peso eco` Se normalizan y se envian al backend como `objective_time`, `objective_risk`, `objective_eco`.

6.10 Reentreno IA (estado de modelo)

El dashboard incorpora operativa de reentreno en cabecera:

- Boton `Reentrenar IA`.
- Bloque de calendario sobre el boton:
  - `Ultimo reentreno`
  - `Siguiente programado`
  - visualizado en `Europe/Madrid`.
- Estado de ejecucion (`idle`, `running`, `done`, `error`) con duracion y timestamp de fin.
- Panel de recomendacion de reentreno con score de deriva (`0-100`) y motivos explicativos.
- Bloque de modelos IA en dos paneles:
  - panel izquierdo: modelo en uso (`EN USO`), candidato elegido (`A/B/C`) y comparativa RMSE,
  - panel derecho: descripcion funcional de los 3 candidatos en columna unica.

Endpoints asociados:

- `POST /api/ml/retrain`:

  - dispara reentreno asincrono (por defecto comando `docker exec spark-client /opt/spark-app/run-ba`
  - devuelve `202 Accepted` si arranca o `409 Conflict` si ya hay uno en curso (incluido curso externo via Airflow).
- `GET /api/ml/retrain/status`:
  - devuelve estado runtime del proceso,
  - devuelve recomendacion cacheada de reentreno y metrica de deriva,
  - devuelve `model_info` con candidatos, criterio de seleccion y ultimo ganador persistido,
  - devuelve `schedule_info` (`last_retrain_at`, `next_scheduled_at`, `timezone`).

Reglas operativas de recomendacion:

- Histeresis con dos umbrales (`RETRAIN_RECOMMEND_ON_THRESHOLD`, `RETRAIN_RECOMMEND_OFF_THRESHOLD`).
- Enfriamiento tras exito (`RETRAIN_COOLDOWN_HOURS`) para evitar reentrenos consecutivos.
- Persistencia de estado y recomendacion en Cassandra (`transport.model_retrain_state`).

Comportamiento del job de reentreno (Spark batch):

- Entrena y compara 3 candidatos (`baseline_rf`, `tuned_baseline_rf`, `enhanced_rf`).
- `enhanced_rf` incluye features de clima y congestion alineadas por `warehouse_id` + ventana de 15 minutos para mejorar prediccion en condiciones operativas.
- Selecciona automaticamente el de menor RMSE para persistir el modelo final.
- El backend persiste el ultimo ganador (`last_selected_model`) y los RMSE de referencia en Cassandra para mostrarlos en cabecera tras reinicios.
- Esto evita que un experimento de features degrade prediccion en produccion del dashboard.

6.11 Concurrencia de reentreno (Airflow + Dashboard)

- El DAG mensual `logistics_kdd_monthly_maintenance` opera con `max_active_runs=1`.
- Si existe un reentreno en curso marcado por Airflow, el boton manual del dashboard no dispara otro (respuesta `409`).
- El estado runtime compartido permite reflejar en cabecera que el modelo esta `reentrenando` aunque el trigger venga desde DAG.

6.12 Reglas de interpretacion de ruta por vehiculo

El dashboard prioriza para cada vehiculo:

1. tramo actual `planned_origin/planned_destination` (estado del generador en `.vehicle_path_state.json`).
2. ruta completa orientada `planned_route_nodes` y `planned_route_label` (origen -> intermedios -> final).
3. Si no existe plan, inferencia por posicion y heading.

Esto evita incoherencias visuales (ejemplo: rutas tipo `X -> X` o proyecciones a nodos no esperados).

Comportamiento visual asociado:

- Tabla de flota y panel de vehiculo muestran la ruta completa cuando existe.

- En mapa de tiempo real, la proyeccion del vehiculo seleccionado dibuja la ruta restante para evitar retrocesos visuales (fallback a siguiente nodo si no hay plan completo).

6.13 ETA en panel de vehiculo

La ETA se calcula en frontend por vehiculo como:

1. distancia al siguiente nodo estimado,
2. velocidad actual (`speed_kmh`),
3. buffer parcial de delay (`delay_minutes`).

Para estabilidad visual existe suavizado temporal, pero con resincronizacion forzada en estos casos:

- cambio de destino estimado (`destinationId`),
- divergencia grande entre ETA cacheado y ETA fisico (`distance/speed + delay`).

Representacion en UI:

- Tabla de vehiculos:
  - `ETA nodos restantes`: ETA por nodos pendientes en orden de paso (preview compacto).
  - `ETA destino final`: ETA al ultimo nodo de la ruta planificada.
- Historial/vehiculo seleccionado (panel):
  - `ETA siguiente nodo`.
  - `ETA nodos restantes` (detalle completo por nodo pendiente).
  - `ETA destino final`.

Con esto se evita que persistan ETAs irreales tras saltos de posicion, cambios de ruta o refrescos incompletos, y se mejora la visibilidad del avance real hasta el destino final.

6.14 Datos y nodos actuales

Red actual (15 nodos): `ACO, ALM, BCN, BIO, CAC, GIJ, LIS, MAD, MAL, MUR, OPO, SEV, VAL, VLL, ZAR`.

Flota actual (15 vehiculos):

- Activos: `14`
- Mantenimiento: `1` (`TRUCK-004`)

Archivos fuente:

- `data/master/warehouses.csv`
- `data/graph/vertices.csv`
- `data/graph/edges.csv`
- `scripts/gps_generator.py`

6.15 Reinicio limpio de demo

Para empezar de cero y evitar sesgo por historico antiguo:

```
./scripts/reset_demo_data.sh
```

Incluye:

- limpieza de eventos locales en `nifi/input` y raw weather/failures,
- reset de estado streaming (Hive + checkpoints curated),
- recreacion de `gps-generator` y `raw-hdfs-loader`.

Modo agresivo (tambien limpia raw HDFS `/data/raw/nifi`):

```
./scripts/reset_demo_data.sh --hard
```

6.16 Nota de uso

Tras cambios de frontend, hacer recarga fuerte (`Ctrl+F5`) en `http://localhost:8501`.

7 Operaciones del Proyecto KDD

7.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Guia de operaciones y troubleshooting`
- Version: `v1.4`
- Fecha: `04/04/2026`
- Repositorio GitHub: `https://github.com/raulsistemasydesarrollo/KDD-Ingenieria-de-datos`

7.2 Indice

1. Arranque y parada
2. Comandos operativos rapidos (scripts clave)
3. Healthcheck en Airflow
4. Arranque del stack desde Airflow
5. Flujo NiFi
6. Validaciones
7. Politica de almacenamiento
8. Matriz operativa end-to-end
9. Limpieza de estado streaming
10. Limpieza de historico failed en Airflow
11. Zona horaria
12. Vistas Hive en hora Madrid
13. Bitacora operativa
14. Operacion del dashboard (estado final)
15. Troubleshooting dashboard
16. Sync de insights Cassandra -> Hive
17. Reentreno IA del dashboard
18. Preview HDFS en navegador (temporal y riesgos)

Este documento resume los comandos recomendados para operar el entorno y validar resultados.

Manual de usuario integral relacionado:

- `docs/manual-usuario.md`

7.3 Arranque y parada

Arrancar todo el stack:

```
./scripts/start_kdd.sh
```

El script de arranque incluye automaticamente:

1. bootstrap de NiFi,
2. `ensure_hive_streaming_compat.sh` (recreacion idempotente de tablas/vistas streaming Hive),
3. sanity check Hive streaming (tablas + vistas Madrid + `COUNT(*)` de ambas vistas),
4. repoblado de `transport_analytics.weather_observations_streaming` desde Cassandra si `v_weather_observations_madrid` queda en `0` filas.

Parar todo el stack:

```
./scripts/stop_kdd.sh
```

7.4 Comandos operativos rapidos (scripts clave)

7.4.1 Reinicio completo o parcial de servicios

Reinicio completo (recomendado para demos):

```
./scripts/stop_kdd.sh
./scripts/start_kdd.sh
```

Reinicio de servicio puntual:

```
docker compose restart dashboard
docker compose restart nifi
docker compose restart spark-client
```

Recrear servicio con nueva imagen/config:

```
sg docker -c "docker compose up -d --build --force-recreate dashboard"
```

7.4.2 Checks de salud de la plataforma

Health HTTP de dashboard:

```
curl -fsS http://localhost:8501/health
```

Estado general de contenedores:

```
docker compose ps
```

Healthcheck Airflow (API interna):

```
curl -fsS http://localhost:8080/health
```

Estado de topics Kafka:

```
docker compose exec -T kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 --list
```

Comprobacion de rutas HDFS:

```
docker compose exec -T hadoop hdfs dfs -ls -R /data/raw/nifi | tail -n 40
docker compose exec -T hadoop hdfs dfs -ls -R /data/curated | tail -n 60
```

7.4.3 Comprobaciones y validaciones por script

Validacion E2E de Hive (batch + streaming):

```
./scripts/validate_hive_pipeline.sh
```

Validacion de parseo meteo (number/string -> double):

```
./scripts/validate_weather_parsing_fix.sh
```

Reparacion idempotente de objetos streaming en Hive:

```
./scripts/ensure_hive_streaming_compat.sh
```

Repoblado meteo en Hive desde Cassandra:

```
./scripts/repopulate_hive_weather_from_cassandra.sh
```

7.4.4 Consultas BBDD (Hive y Cassandra)

Hive: tablas y recuentos clave:

```
docker compose exec -T spark-client spark-sql -e "SHOW TABLES IN transport_analytics;"
docker compose exec -T spark-client spark-sql -e "SELECT COUNT(*) AS delay_rows FROM transport_a
docker compose exec -T spark-client spark-sql -e "SELECT COUNT(*) AS weather_rows FROM transport
```

Cassandra: estado de flota, meteo e insights:

```
docker compose exec -T cassandra cqlsh -e "SELECT vehicle_id, warehouse_id, route_id, last_event
docker compose exec -T cassandra cqlsh -e "SELECT bucket, weather_timestamp, weather_event_id, w
docker compose exec -T cassandra cqlsh -e "SELECT bucket, entity_type, profile, min_congestion,
```

7.4.5 Troubleshooting rapido

Limpiar estado streaming (tablas + checkpoints) y recomponer vistas:

```
./scripts/reset_streaming_state.sh
```

Reset de demo (soft / hard):

```
./scripts/reset_demo_data.sh
./scripts/reset_demo_data.sh --hard
```

Limpieza de runs `failed` en Airflow:

```
./scripts/cleanup_airflow_failed_runs.sh
./scripts/cleanup_airflow_failed_runs.sh --apply
```

Reconstruir flujo NiFi y limpiar PGs legacy:

```
./scripts/bootstrap_nifi_flow.sh
./scripts/cleanup_nifi_legacy_pgs.py
```

Preview HDFS desde navegador (sin tocar HDFS interno):

```
./scripts/toggle_hdfs_browser_preview_hosts.sh status
sudo ./scripts/toggle_hdfs_browser_preview_hosts.sh enable
```

7.5 Healthcheck en Airflow

Captura de la vista operativa de Airflow:

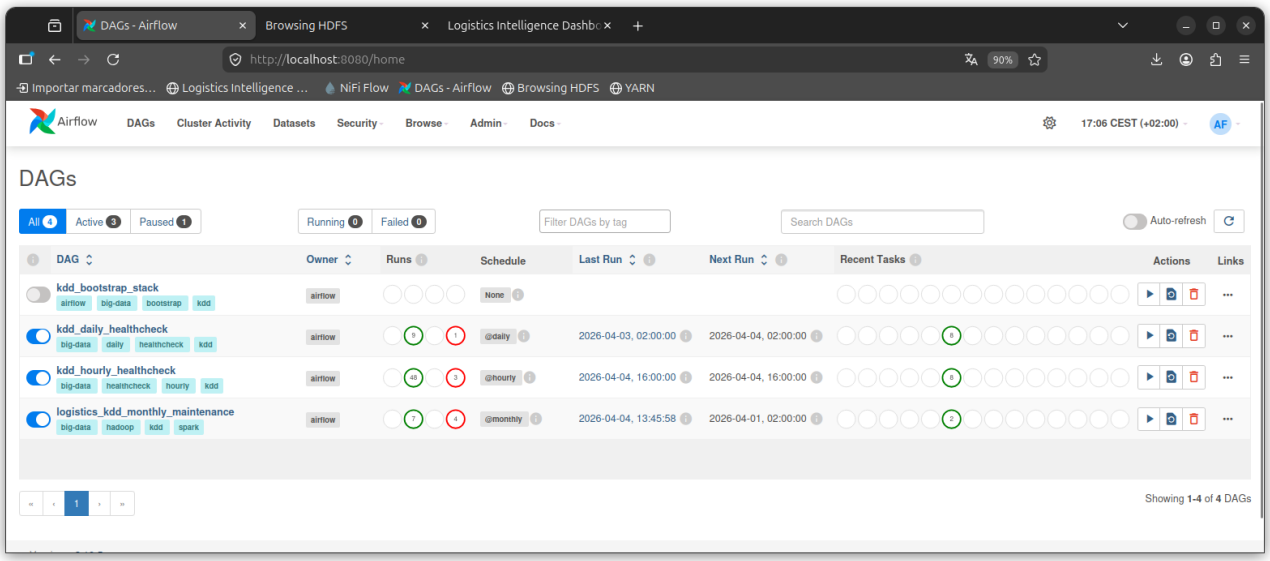

Figure 9: Airflow

Se incorporan dos DAGs de comprobacion:

- `kdd_daily_healthcheck`
- `kdd_hourly_healthcheck`

Estos DAGs validan:

1. Contenedores core activos (`kafka`, `hadoop`, `hive-metastore`, `nifi`, `spark-client`).
2. Topics Kafka necesarios (`transport.filtered`, `transport.weather.filtered`).
3. Paths base de HDFS.
4. Tablas Hive base de `transport_analytics`.
5. Vistas en hora Madrid:
   - `transport_analytics.v_weather_observations_madrid`
   - `transport_analytics.v_delay_metrics_streaming_madrid`

6. Consulta smoke de ambas vistas (con degradacion segura):
   - si las tablas streaming fuente no estan disponibles temporalmente, el smoke no rompe el DAG y valida metastore/batch.
7. Auto-recuperacion meteo:
   - tarea `repopulate_hive_weather_if_empty`,
   - si `weather_rows=0` en `v_weather_observations_madrid`, ejecuta `./scripts/repopulate_hive_weathe` antes del smoke query de meteo.

Alertas:

- Cada DAG incluye `on_failure_callback` (alerta en logs de Airflow).
- Si defines `AIRFLOW_ALERT_EMAIL`, se habilita envio de email en fallo.

7.6 Arranque del stack desde Airflow

DAG de bootstrap:

- `kdd_bootstrap_stack`

Este DAG arranca servicios no-Airflow del stack (`kafka`, `nifi`, `gps-generator`, `hadoop`, `raw-hdfs-loader`, `postgres-metastore`, `hive-metastore`, `cassandra`, `spark-client`, `dashboard`) y muestra estado final.

Script recomendado (arranca Airflow y dispara el DAG):

```
./scripts/start_airflow_then_stack.sh
```

7.7 Flujo NiFi

Bootstrap automatico del flujo de ingesta:

```
./scripts/bootstrap_nifi_flow.sh
```

Estructura visual creada por defecto:

- Process Group: `kdd_ingestion_auto_v9`
- Subgrupos:
  - `gps_ingestion`
  - `weather_ingestion`
- Archivado GPS:
  - `SplitText` + `UpdateAttribute` + `PutFile` con filename unico por split (`${filename}_${fragment.index`

Limpieza de grupos legacy (para dejar solo la version actual):

```
./scripts/cleanup_nifi_legacy_pgs.py
```

7.8 Validaciones

Validacion E2E principal (batch + streaming):

```
./scripts/validate_hive_pipeline.sh
```

Validacion especifica del parseo meteo (soporta numeros y numeros como string):

```
./scripts/validate_weather_parsing_fix.sh
```

7.9 Politica de almacenamiento

El proyecto sigue el criterio recomendado:

- Capa `raw`: conservar entrada original en `JSON/JSONL` para trazabilidad y reprocesado.
- Capa `curated`: persistir salida transformada en `Parquet` para analitica eficiente.

Rutas principales:

- Raw en HDFS:
  - `/data/raw/gps_events.jsonl`
  - `/data/raw/nifi/...`
- Curated en HDFS:
  - `/data/curated/enriched_events`
  - `/data/curated/delay_metrics_streaming`
  - `/data/curated/weather_observations_streaming`
  - `/data/curated/enriched_events_streaming`

Comprobacion de rutas y formatos:

```
sg docker -c "docker compose exec -T hadoop hdfs dfs -ls -R /data/raw | head -n 50"
sg docker -c "docker compose exec -T hadoop hdfs dfs -ls -R /data/curated | head -n 100"
```

Comprobacion SQL en Hive:

```
sg docker -c "docker compose exec -T spark-client spark-sql -e \"SHOW TABLES IN transport_analyt
sg docker -c "docker compose exec -T spark-client spark-sql -e \"DESCRIBE FORMATTED transport_an
```

7.10 Consultas desde terminal (Hive y Cassandra)

7.10.1 Hive (via spark-sql en `spark-client`)

Listar tablas del esquema:

```
docker compose exec -T spark-client spark-sql -e "SHOW TABLES IN transport_analytics;"
```

Ver estructura de una tabla:

```
docker compose exec -T spark-client spark-sql -e "DESCRIBE transport_analytics.delay_metrics_str
```

Consultar ultimos registros (primero valida que haya filas):

```
docker compose exec -T spark-client spark-sql -e "SELECT COUNT(*) AS weather_rows FROM transport
docker compose exec -T spark-client spark-sql -e "SELECT weather_event_id, weather_timestamp_utc
```

Si `weather_rows = 0`, la vista existe pero no hay carga streaming meteo en Hive en ese momento.
Como consulta operativa inmediata usa Cassandra:

```
docker compose exec -T cassandra cqlsh -e "SELECT bucket, weather_timestamp, weather_event_id, w
```

Para repoblar Hive meteo desde Cassandra:

```
./scripts/repopulate_hive_weather_from_cassandra.sh
```

Comprobar si existe la tabla streaming fisica (puede no existir y seguir siendo correcto en este proyecto):

```
docker compose exec -T spark-client spark-sql -e "SHOW TABLES IN transport_analytics LIKE 'weath
```

Reparar/asegurar objetos streaming de Hive (tablas + vistas Madrid) tras limpiezas:

```
./scripts/ensure_hive_streaming_compat.sh
```

Sanity check manual (equivalente al que ejecuta `start_kdd.sh`):

```
docker compose exec -T spark-client spark-sql -e "
SHOW TABLES IN transport_analytics LIKE 'delay_metrics_streaming';
SHOW TABLES IN transport_analytics LIKE 'enriched_events_streaming';
SHOW TABLES IN transport_analytics LIKE 'weather_observations_streaming';
SHOW TABLES IN transport_analytics LIKE 'v_delay_metrics_streaming_madrid';
SHOW TABLES IN transport_analytics LIKE 'v_weather_observations_madrid';
SELECT COUNT(*) AS delay_rows FROM transport_analytics.v_delay_metrics_streaming_madrid;
SELECT COUNT(*) AS enriched_stream_rows FROM transport_analytics.enriched_events_streaming;
SELECT COUNT(*) AS weather_rows FROM transport_analytics.v_weather_observations_madrid;
"
```

7.10.2 Cassandra (via `cqlsh` en contenedor `cassandra`)

Listar keyspaces:

```
docker compose exec -T cassandra cqlsh -e "DESCRIBE KEYSPACES;"
```

Listar tablas del keyspace `transport`:

```
docker compose exec -T cassandra cqlsh -e "DESCRIBE TABLES IN transport;"
```

Ver definicion de tabla:

```
docker compose exec -T cassandra cqlsh -e "DESCRIBE TABLE transport.vehicle_latest_state;"
```

Consultar estado de flota:

```
docker compose exec -T cassandra cqlsh -e "SELECT vehicle_id, warehouse_id, route_id, last_event
```

Consultar clima reciente:

```
docker compose exec -T cassandra cqlsh -e "SELECT weather_timestamp, warehouse_id, temperature_c
```

Consultar snapshots de insights de red:

```
docker compose exec -T cassandra cqlsh -e "SELECT bucket, entity_type, profile, min_congestion,
```

7.11 Sync de insights Cassandra -> Hive

Para consolidar snapshots de insights en Hive:

```
sg docker -c "docker compose exec -T spark-client /opt/spark-app/run-insights-sync.sh"
```

Tablas resultantes en Hive:

- `transport_analytics.network_insights_snapshots_hive`
- `transport_analytics.network_insights_hourly_trends`

Validacion rapida:

```
docker compose exec -T spark-client spark-sql -e "SELECT COUNT(*) AS c FROM transport_analytics.
docker compose exec -T spark-client spark-sql -e "SELECT snapshot_hour, profile, min_congestion,
```

Si aparece error `permission denied` al ejecutar `run-insights-sync.sh`:

```
chmod +x spark-app/run-insights-sync.sh
sg docker -c "docker compose up -d --build spark-client"
```

7.12 Matriz operativa end-to-end

Tabla de referencia rapida para trazar cada flujo desde origen hasta consumo analitico:

Flujo	Fuente	Kafka	HDFS	Hive
GPS raw	`gps-generator` -> NiFi (`nifi/input/*.jsonl`)	`transport.raw`	`/data/raw/nifi/gps`	No aplica (capa raw)
GPS filtrado/normalizado	NiFi	`transport.filtered`	`/data/curated/delay` (fallback parquet streaming)	`transport_analytics.delay_m`
GPS enriquecido streaming	Spark streaming	`transport.filtered`	`/data/curated/enri` (fallback parquet streaming)	`transport_analytics.enriche`
Weather raw	API Open-Meteo -> NiFi	`transport.weather.raw`	`/data/raw/nifi/weather`	No aplica (capa raw)
Weather filtrado/normalizado	NiFi	`transport.weather.filtered`	`/data/curated/weat` (fallback parquet streaming)	`transport_analytics.weather` (vista operativa)
Batch historico GPS	HDFS seed/raw (`/data/raw/gps_events.jsonl`)	No aplica	`/data/curated/enri`	`transport_analytics.enriche` `transport_analytics.delay_m` `transport_analytics.route_g`
Grafo shortest path	`data/graph/*` (batch Spark)	No aplica	No aplica (tabla Hive)	`transport_analytics.route_s`
ML riesgo retraso	`enriched_events` (batch Spark ML)	No aplica	`hdfs://hadoop:9000/models/delay_risk_ml` (modelo)	`transport_analytics.ml_dela`

Validacion ML (batch/reentreno) recomendada:

```
sg docker -c "docker compose exec -T spark-client bash /opt/spark-app/run-batch.sh"
sg docker -c "docker compose exec -T hadoop hdfs dfs -du -h /models/delay_risk_rf"
```

Salida esperada (iteracion 04/04/2026):

- linea en logs con comparativa:
  - `INFO: ML A/B delay risk => baseline_rmse=... | tuned_baseline_rmse=... | enhanced_rmse=... | selected=...`
- modelo persistido en `hdfs://hadoop:9000/models/delay_risk_rf` con tamano ~`1.2 MB` (estado validado 04/04/2026).

Checklist de comprobacion rapida:

```
sg docker -c "docker compose exec -T kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server kaf
sg docker -c "docker compose exec -T hadoop hdfs dfs -ls -R /data/raw/nifi | tail -n 40"
sg docker -c "docker compose exec -T hadoop hdfs dfs -ls -R /data/curated | tail -n 60"
sg docker -c "docker compose exec -T spark-client spark-sql -e \"SHOW TABLES IN transport_analyt
```

7.13 Limpieza de estado streaming

Elimina tablas de streaming en Hive y checkpoints en HDFS para arrancar limpio:

```
./scripts/reset_streaming_state.sh
```

7.14 Limpieza de historico failed en Airflow

Para limpiar runs fallidos antiguos (sin tocar runs `success`):

```
./scripts/cleanup_airflow_failed_runs.sh
```

Aplicar borrado:

```
./scripts/cleanup_airflow_failed_runs.sh --apply
```

Estado esperado tras limpieza:

- Sin `failed` para:
  - `kdd_daily_healthcheck`
  - `kdd_hourly_healthcheck`

Comprobacion SQL directa en metastore de Airflow:

```
docker exec airflow-postgres psql -U airflow -d airflow -c "
SELECT dag_id, count(*) AS failed_runs
FROM dag_run
WHERE state='failed'
  AND dag_id IN ('kdd_daily_healthcheck','kdd_hourly_healthcheck')
GROUP BY dag_id;"
```

7.15 Zona horaria

- Spark usa por defecto `SPARK_SQL_TIMEZONE=Europe/Madrid`.
- Los eventos entran habitualmente con marca temporal UTC (`...Z`), por lo que puede ser util consultar tanto UTC como Madrid.

7.16 Vistas Hive en hora Madrid

Vistas disponibles:

- `transport_analytics.v_weather_observations_madrid`
- `transport_analytics.v_delay_metrics_streaming_madrid`

Si una vista falla con `TABLE_OR_VIEW_NOT_FOUND`, ejecutar:

```
./scripts/ensure_hive_streaming_compat.sh
```

Consulta ejemplo meteo:

```sql
SELECT weather_event_id, weather_timestamp_utc, weather_timestamp_madrid
FROM transport_analytics.v_weather_observations_madrid
ORDER BY weather_timestamp_utc DESC
LIMIT 10;
```

Consulta ejemplo GPS agregado:

```sql
SELECT warehouse_id, window_start_utc, window_start_madrid, window_end_utc, window_end_madrid
FROM transport_analytics.v_delay_metrics_streaming_madrid
ORDER BY window_start_utc DESC
LIMIT 10;
```

7.17 Bitacora operativa (actualizada 04/04/2026)

Cambios aplicados:

1. Healthchecks Airflow estabilizados frente a ausencia temporal de tablas streaming fuente.
2. Historial rojo de healthchecks limpiado en Airflow.
3. NiFi reorganizado por dominios (`gps_ingestion` y `weather_ingestion`) en `kdd_ingestion_auto_v9`.
4. Process Group legacy eliminado para evitar ruido visual en la UI de NiFi.
5. Incorporado flujo de insights de red con persistencia en Cassandra y sync a Hive.
6. Reentreno mensual endurecido contra paralelismo (`max_active_runs=1` en DAG mensual).
7. Dashboard muestra `Ultimo reentreno` y `Siguiente programado` en cabecera (hora Madrid).
8. Trigger manual de reentreno bloqueado cuando existe ejecucion activa desde Airflow (`409`).
9. Guia operativa de preview HDFS actualizada con opcion segura por `/etc/hosts` y riesgos del cambio temporal en `hdfs-site.xml`.

7.18 Nota de tablas en tiempo real

- `transport_analytics.enriched_events` es historico batch.

- Para consultas en vivo de eventos enriquecidos usar `transport_analytics.enriched_events_streaming`.

7.19 Operacion del dashboard (estado final)

7.19.1 Desacople de vistas

El dashboard opera con dos contextos de filtro independientes:

1. `Operacion en Tiempo Real` (izquierda):
   - filtros: `Origen RT`, `Destino RT`.
2. `Analisis de Red Logistica` (derecha):
   - filtros: `Origen`, `Destino`, `Perfil`.

Un cambio de filtro en un bloque no debe alterar el otro.

Adicionalmente, en el estado actual:

1. Los selectores de origen/destino en ambas vistas se cargan ordenados alfabeticamente.
2. Todas las tablas del dashboard admiten ordenacion por columna.
3. Perfiles de ruta disponibles: `balanced`, `fastest`, `resilient`, `eco`, `low_risk`, `reliable`.
4. Controles de optimizacion activos:
   - pesos `tiempo/riesgo/eco`,
   - `Patron horario` (`auto`, `peak`, `offpeak`, `night`),
   - `Evitar nodos`.

7.19.2 Reglas de filtro en Tiempo Real

- `TODOS -> TODOS`: muestra toda la flota.
- `TODOS -> X`: muestra vehiculos cuyo destino reportado es `X`.
- `X -> TODOS`: muestra vehiculos cuyo origen reportado es `X`.
- `X -> Y`: restringe por corredor y coherencia espacial.

7.19.3 Fuente de ruta por vehiculo

Para evitar inconsistencias de inferencia:

1. Se usa plan de trayecto por vehiculo desde `nifi/input/.vehicle_path_state.json`.
2. Ese plan se expone por API en cada item como:
   - `planned_origin`
   - `planned_destination`
   - `planned_route_nodes`
   - `planned_route_label`
3. El frontend prioriza estos campos para `Ruta reportada`, `Siguiente nodo` y linea de proyeccion de ruta restante.

7.20 Troubleshooting dashboard

7.20.1 Caso: veo ruta incoherente (`X -> X`) o destino inesperado

1. Forzar recarga de navegador:

```
# navegador
Ctrl+F5
```

2. Comprobar payload de vehiculos:

```
docker compose exec -T dashboard python -c "import json,urllib.request;d=json.load(urllib.reques
```

3. Verificar estado del plan del generador:

```
python3 - <<'PY'
import json
with open('nifi/input/.vehicle_path_state.json','r',encoding='utf-8') as f:
    d=json.load(f)
print(d.get('V1'))
PY
```

Si `planned_origin/planned_destination` o `planned_route_nodes` no aparecen en API, reiniciar dashboard:

```
docker compose restart dashboard
```

7.20.2 Caso: ETA incoherente (ejemplo: >300 km y ETA en pocos minutos)

Comprobar primero distancia y velocidad mostradas en el panel de vehiculo. Si son coherentes entre si pero la ETA no lo es:

1. Forzar recarga fuerte del frontend (`Ctrl+F5`) y pulsar `Actualizar` en el dashboard.
2. Verificar que el frontend ya incluye el ajuste de resincronizacion de ETA:

```
rg -n "forceResync|divergence|destinationChanged" dashboard/static/app.js
```

3. Reiniciar el servicio dashboard para limpiar estado en memoria:

```
docker compose restart dashboard
```

Nota tecnica: - El calculo ETA ahora fuerza resincronizacion inmediata cuando hay cambio de destino estimado o desviacion grande entre ETA cacheado y ETA fisico (`distance/speed + delay`), evitando ETAs irreales persistentes.

7.21 Reentreno IA del dashboard

El dashboard expone operativa de reentreno de modelo desde la cabecera.

Bloques de cabecera en estado actual:

1. Panel izquierdo: `EN USO`, candidato elegido (`A/B/C`) y comparativa RMSE.
2. Panel derecho: descripcion de los 3 candidatos en columna unica.

3. Bloque de calendario sobre el boton:
   - `Ultimo reentreno`
   - `Siguiente programado`
   - mostrado en `Europe/Madrid`.

7.21.1 Endpoints

- Trigger manual:

```
curl -s -X POST http://localhost:8501/api/ml/retrain \
  -H 'Content-Type: application/json' \
  -d '{"trigger":"manual_dashboard"}'
```

- Estado + recomendacion:

```
curl -s http://localhost:8501/api/ml/retrain/status
```

Respuesta ampliada:

- `state`
- `advice`
- `model_info`
- `schedule_info` (`last_retrain_at`, `next_scheduled_at`, `timezone`)

7.21.2 Variables de entorno relevantes

- `RETRAIN_COMMAND` (por defecto: `docker exec spark-client /opt/spark-app/run-batch.sh`)
- `RETRAIN_TIMEOUT_SECONDS`
- `RETRAIN_RECOMMEND_THRESHOLD`
- `RETRAIN_RECOMMEND_ON_THRESHOLD`
- `RETRAIN_RECOMMEND_OFF_THRESHOLD`
- `RETRAIN_COOLDOWN_HOURS`
- `RETRAIN_STATUS_POLL_CACHE_SECONDS`
- `CASSANDRA_RETRAIN_STATE_TABLE` (por defecto: `model_retrain_state`)

7.21.3 Persistencia Cassandra

Tabla de estado de recomendacion/reentreno:

```
docker compose exec -T cassandra cqlsh -e "SELECT model_name, last_success_at, last_run_at, last
```

7.21.4 Concurrencia y bloqueo de triggers

1. El DAG mensual `logistics_kdd_monthly_maintenance` opera con `max_active_runs=1`.
2. Si existe reentreno en curso (dashboard o Airflow), `POST /api/ml/retrain` devuelve `409`.
3. El estado compartido permite que la cabecera refleje `running` cuando el trigger viene desde Airflow.

7.22 Preview HDFS en navegador (temporal y riesgos)

Contexto:

- En NameNode UI (`http://localhost:9870`) puede aparecer `Couldn't preview the file`.
- Suele deberse a redirect a `http://hadoop:9864/...` que el host no resuelve.

Camino recomendado (estable):

```
sudo ./scripts/toggle_hdfs_browser_preview_hosts.sh enable
```

Revertir:

```
sudo ./scripts/toggle_hdfs_browser_preview_hosts.sh disable
```

Implicaciones del recomendado:

1. Solo modifica `/etc/hosts` del host.
2. No toca `hdfs-site.xml`.
3. No afecta Spark/Airflow internos.

Cambio temporal alternativo (solo prueba puntual, no recomendado):

1. Ajustar `docker/hadoop/conf/hdfs-site.xml` con:
   - `dfs.datanode.hostname=localhost`
   - `dfs.client.use.datanode.hostname=true`
   - `dfs.datanode.use.datanode.hostname=true`
2. Rebuild/recreate de `hadoop`.

Implicaciones del alternativo:

1. Puede mejorar preview web desde host.
2. Puede romper escrituras HDFS desde contenedores Spark/Airflow:
   - `Connection refused`
   - `could only be written to 0 of the 1 minReplication nodes`
3. Debe revertirse tras la prueba.

8 Manual de Usuario - Plataforma KDD Logistica

8.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Manual de usuario funcional y operativo`
- Version: `v1.1`
- Fecha: `04/04/2026`
- Repositorio GitHub: `https://github.com/raulsistemasydesarrollo/KDD-Ingenieria-de-datos`

8.2 Indice

1. Objetivo y alcance

2. Requisitos previos
3. Arranque y parada del stack
4. Uso completo del dashboard
5. Uso de DAGs en Airflow
6. Comprobaciones de salud y estado
7. Consultas de datos (Hive y Cassandra)
8. Troubleshooting guiado
9. Comandos de referencia rapida

8.3 1. Objetivo y alcance

Este manual explica, de forma practica, como usar toda la aplicacion:

1. Arrancar y parar la plataforma completa.
2. Operar el dashboard en sus dos vistas (Tiempo Real y Red Logistica).
3. Entender y ejecutar los DAGs de Airflow.
4. Verificar salud de servicios y datos.
5. Resolver incidencias operativas comunes.

8.4 2. Requisitos previos

Entorno recomendado:

1. Docker y Docker Compose activos.
2. Usuario con permisos para ejecutar Docker (grupo `docker`).
3. Proyecto clonado en local.
4. Puertos disponibles:
   - `8080` (Airflow)
   - `8443` (NiFi)
   - `8501` (Dashboard)
   - `9870` (HDFS UI)

Comprobar Docker:

```
docker ps
```

8.5 3. Arranque y parada del stack

8.5.1 3.1 Arranque recomendado (todo en uno)

```
./scripts/start_kdd.sh
```

Que hace este script:

1. Levanta el stack con `docker compose up -d --build`.
2. Espera disponibilidad de NiFi.
3. Ejecuta bootstrap del flujo NiFi (`kdd_ingestion_auto_v9`).
4. Asegura compatibilidad de objetos Hive streaming.

5. Ejecuta sanity check de tablas/vistas en Hive.
6. Si la vista meteo en Hive esta vacia, repuebla desde Cassandra.
7. Espera health del dashboard y muestra estado final.

8.5.2 3.2 Arranque alternativo (Airflow primero)

```
./scripts/start_airflow_then_stack.sh
```

Este flujo:

1. Levanta solo Airflow.
2. Espera `http://localhost:8080/health`.
3. Despausa y dispara el DAG `kdd_bootstrap_stack`.
4. Deja trazabilidad del arranque en Airflow.

8.5.3 3.3 Parada completa

```
./scripts/stop_kdd.sh
```

8.5.4 3.4 Reinicio rapido por servicio

```
docker compose restart dashboard
docker compose restart nifi
docker compose restart spark-client
```

8.5.5 3.5 Hard reset de demo

Soft reset (limpia estado streaming y eventos locales):

```
./scripts/reset_demo_data.sh
```

Hard reset (incluye limpieza raw historico en HDFS):

```
./scripts/reset_demo_data.sh --hard
```

8.6 4. Uso completo del dashboard

URL:

- `http://localhost:8501`

Health endpoint:

```
curl -fsS http://localhost:8501/health
```

8.6.1 4.1 Estructura funcional del dashboard

El dashboard esta dividido en dos contextos independientes:

1. `Operacion en Tiempo Real` (columna izquierda).
2. `Analisis de Red Logistica` (columna derecha).

Regla clave:

- Los filtros de una vista no afectan a la otra.

8.6.2 4.2 Vista de Tiempo Real (izquierda)

Elementos principales:

1. Mapa de vehiculos en vivo.
2. Trazas recientes (lineas naranjas).
3. Proyeccion de ruta restante del vehiculo seleccionado (linea amarilla discontinua).
4. Tabla de flota con ETA por nodos restantes y ETA destino final.
5. Panel de detalle/historial de vehiculo.

Filtros de esta vista:

1. `Origen RT`
2. `Destino RT`

Casos tipicos de filtro:

1. `TODOS -> TODOS`: vista global de flota.
2. `TODOS -> X`: vehiculos con destino `X`.
3. `X -> TODOS`: vehiculos con origen `X`.
4. `X -> Y`: corredor concreto.

8.6.3 4.3 Vista de Red Logistica (derecha)

Elementos principales:

1. Grafo de almacenes y conexiones.
2. Ruta calculada resaltada en rojo.
3. Tabla de ruta con distancia, tiempo base, penalizacion meteo y ETA final.
4. Bloque `Insights de red` con rankings de cuellos de botella y nodos criticos.

Filtros de esta vista:

1. `Origen`
2. `Destino`
3. `Perfil`
4. `Patron horario`
5. `Peso tiempo`, `Peso riesgo`, `Peso eco`
6. `Evitar nodos`

Perfiles de optimizacion disponibles:

1. `balanced`
2. `fastest`
3. `resilient`
4. `eco`

5. `low_risk`
6. `reliable`

8.6.4 4.4 Flujo recomendado de uso del dashboard

1. Verifica que el banner/metricas cargan y que no hay error de fuente.
2. En Tiempo Real, selecciona un vehiculo y valida ruta reportada + ETA.
3. En Red Logistica, define origen/destino y pulsa calcular mejor ruta.
4. Ajusta perfil, patron horario y pesos para comparar alternativas.
5. Usa `Evitar nodos` para simular cortes/incidencias.
6. Revisa bloque de `Insights` para priorizacion operativa.

8.6.5 4.5 Diagnostico de fuentes del dashboard

Endpoint:

```
curl -s http://localhost:8501/api/debug/sources
```

Este endpoint informa, entre otros:

1. Fuente activa de vehiculos (`cassandra` o `nifi_files`).
2. Fuente activa de clima (`cassandra` o `nifi_raw_archive`).
3. Estado de conexion Cassandra.
4. Conteo de nodos/aristas cargados.

8.6.6 4.6 Reentreno IA desde dashboard

Trigger manual:

```
curl -s -X POST http://localhost:8501/api/ml/retrain \
  -H 'Content-Type: application/json' \
  -d '{"trigger":"manual_dashboard"}'
```

Estado y recomendacion:

```
curl -s http://localhost:8501/api/ml/retrain/status
```

Comportamiento:

1. Si ya hay reentreno en curso (dashboard o Airflow), responde `409`.
2. Si arranca correctamente, responde `202`.
3. La cabecera del dashboard muestra estado, recomendacion y modelo en uso.
4. Sobre el boton `Reentrenar IA` se muestran:
   - `Ultimo reentreno`
   - `Siguiente programado` en horario `Europe/Madrid`.

8.7 5. Uso de DAGs en Airflow

URL:

- `http://localhost:8080`

Credenciales por defecto:

1. Usuario: `airflow`
2. Password: `airflow`

8.7.1 5.1 DAG `kdd_bootstrap_stack`

Tipo:

- Arranque bajo demanda (`schedule=None`).

Para que sirve:

1. Arranca servicios no-Airflow del stack.
2. Ejecuta `ensure_hive_streaming_compat.sh`.
3. Publica estado final con `docker compose ps`.

Tareas:

1. `start_non_airflow_stack`
2. `ensure_hive_streaming_compat`
3. `show_stack_status`

8.7.2 5.2 DAG `kdd_hourly_healthcheck`

Tipo:

- Ejecucion horaria (`@hourly`).

Para que sirve:

1. Detectar incidencias de infraestructura/datos con baja latencia.

Chequeos:

1. Contenedores core.
2. Topics Kafka de streaming.
3. Paths HDFS (`/data/raw`, `/data/raw/nifi`, `/data/curated`).
4. Tablas Hive base.
5. Vistas Madrid.
6. Smoke query de clima y delay con degradacion segura.
7. Auto-repoblado de meteo en Hive si `weather_rows=0`.

8.7.3 5.3 DAG `kdd_daily_healthcheck`

Tipo:

- Ejecucion diaria (`@daily`).

Para que sirve:

1. Evidencia diaria de salud operativa para seguimiento y entrega.

Chequeos:

- Equivalentes al DAG horario, con foco de control diario.

8.7.4 5.4 DAG `logistics_kdd_monthly_maintenance`

Tipo:

- Ejecucion mensual (`@monthly`).

Para que sirve:

1. Reentreno/recalculo batch de Spark (`run-batch.sh`).
2. Limpieza de checkpoints temporales en HDFS.
3. Evitar solapamiento de runs del propio DAG (`max_active_runs=1`).

Tareas:

1. `retrain_graph_model`
2. `cleanup_temp_hdfs`

8.7.5 5.5 Como ejecutar DAGs manualmente

Desde UI de Airflow:

1. Abre `DAGs`.
2. Activa el DAG (toggle) si esta pausado.
3. Pulsa `Trigger DAG`.
4. Entra al `Graph` o `Grid` para revisar tareas.

Desde CLI (contenedor Airflow):

```
docker compose exec -T airflow-webserver /home/airflow/.local/bin/airflow dags list
docker compose exec -T airflow-webserver /home/airflow/.local/bin/airflow dags trigger kdd_boots
docker compose exec -T airflow-webserver /home/airflow/.local/bin/airflow dags list-runs -d kdd_
```

8.8 6. Comprobaciones de salud y estado

8.8.1 6.1 Estado de servicios

```
docker compose ps
```

8.8.2 6.2 Estado de topics Kafka

```
docker compose exec -T kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 --list
```

8.8.3 6.3 Estado de HDFS

```
docker compose exec -T hadoop hdfs dfs -ls -R /data/raw/nifi | tail -n 40
docker compose exec -T hadoop hdfs dfs -ls -R /data/curated | tail -n 60
```

8.8.4 6.4 Estado de objetos Hive

```
docker compose exec -T spark-client spark-sql -e "SHOW TABLES IN transport_analytics;"
docker compose exec -T spark-client spark-sql -e "SELECT COUNT(*) AS delay_rows FROM transport_a
docker compose exec -T spark-client spark-sql -e "SELECT COUNT(*) AS weather_rows FROM transport
```

8.9 7. Consultas de datos (Hive y Cassandra)

8.9.1 7.1 Consultas Hive utiles

```
docker compose exec -T spark-client spark-sql -e "DESCRIBE transport_analytics.enriched_events_s
docker compose exec -T spark-client spark-sql -e "SELECT warehouse_id, window_start_utc, window_
docker compose exec -T spark-client spark-sql -e "SELECT weather_event_id, weather_timestamp_utc
```

8.9.2 7.2 Consultas Cassandra utiles

```
docker compose exec -T cassandra cqlsh -e "DESCRIBE TABLES IN transport;"
docker compose exec -T cassandra cqlsh -e "SELECT vehicle_id, warehouse_id, route_id, last_event
docker compose exec -T cassandra cqlsh -e "SELECT bucket, weather_timestamp, weather_event_id, w
docker compose exec -T cassandra cqlsh -e "SELECT bucket, entity_type, profile, min_congestion,
```

8.10 8. Troubleshooting guiado

8.10.1 8.1 Dashboard no responde

1. Verifica health:

```
curl -fsS http://localhost:8501/health
```

2. Reinicia servicio:

```
docker compose restart dashboard
```

8.10.2 8.2 Vista meteo vacia en Hive (`weather_rows=0`)

1. Asegura compatibilidad de objetos:

```
./scripts/ensure_hive_streaming_compat.sh
```

2. Repuebla desde Cassandra:

```
./scripts/repopulate_hive_weather_from_cassandra.sh
```

8.10.3 8.3 Estado streaming inconsistente

1. Limpia estado streaming:

```
./scripts/reset_streaming_state.sh
```

2. Arranca de nuevo el stack:

```
./scripts/start_kdd.sh
```

8.10.4 8.4 Flujo NiFi desactualizado o ausente

```
./scripts/bootstrap_nifi_flow.sh
./scripts/cleanup_nifi_legacy_pgs.py
```

8.10.5 8.5 Runs `failed` antiguos en Airflow

```
./scripts/cleanup_airflow_failed_runs.sh
./scripts/cleanup_airflow_failed_runs.sh --apply
```

8.10.6 8.6 ETA/ruta incoherente en frontend

1. Fuerza recarga de navegador (`Ctrl+F5`).
2. Verifica payload de vehiculos:

```
curl -s http://localhost:8501/api/vehicles/latest?limit=5
```

3. Reinicia dashboard si persiste:

```
docker compose restart dashboard
```

8.10.7 8.7 HDFS UI muestra "Couldn't preview the file"

Opcion recomendada (sin tocar HDFS interno):

```
./scripts/toggle_hdfs_browser_preview_hosts.sh status
sudo ./scripts/toggle_hdfs_browser_preview_hosts.sh enable
```

Implicaciones de la opcion recomendada:

1. Solo afecta al host local (`/etc/hosts`).
2. Mantiene estabilidad de Spark/Airflow contra HDFS.
3. Permite que el navegador resuelva `hadoop` al seguir el redirect de NameNode.

Cambio temporal alternativo (solo para prueba puntual, no recomendado):

1. Cambiar `docker/hadoop/conf/hdfs-site.xml` para anunciar DataNode como `localhost`.
2. Reconstruir/recrear `hadoop`.

Implicaciones del cambio temporal alternativo:

1. El preview web puede funcionar mejor desde host.
2. Spark/Airflow en contenedores pueden fallar escribiendo en HDFS (`Connection refused`, `minReplication nodes`).
3. Revertir el cambio al terminar la prueba.

8.11 9. Comandos de referencia rapida

Arranque/parada:

```
./scripts/start_kdd.sh
./scripts/stop_kdd.sh
```

Healthchecks:

```
docker compose ps
curl -fsS http://localhost:8501/health
curl -fsS http://localhost:8080/health
```

Validaciones:

```
./scripts/validate_hive_pipeline.sh
./scripts/validate_weather_parsing_fix.sh
```

Recuperacion:

```
./scripts/ensure_hive_streaming_compat.sh
./scripts/repopulate_hive_weather_from_cassandra.sh
./scripts/reset_streaming_state.sh
./scripts/reset_demo_data.sh --hard
```

Airflow manual:

```
docker compose exec -T airflow-webserver /home/airflow/.local/bin/airflow dags trigger kdd_boots
docker compose exec -T airflow-webserver /home/airflow/.local/bin/airflow dags list-runs -d kdd_
```

9 Arquitectura del Proyecto

9.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Arquitectura tecnica actual`
- Version: `v1.4`
- Fecha: `04/04/2026`
- Repositorio GitHub: `https://github.com/raulsistemasydesarrollo/KDD-Ingenieria-de-datos`

9.2 Indice

1. Vista general
2. Diagrama de arquitectura
3. Componentes y responsabilidades
4. Flujo de datos end-to-end
5. Capas de almacenamiento
6. Decision de desacople en dashboard

9.3 1. Vista general

La plataforma implementa el ciclo KDD completo en local con Docker Compose:

1. Ingesta dual (GPS + clima) en NiFi.
2. Bus de eventos en Kafka (raw y filtered).
3. Persistencia raw en HDFS (sincronizacion desde raw-archive NiFi).

4. Procesamiento batch y streaming con Spark.
5. Explotacion analitica en Hive + estado operativo en Cassandra.
6. Orquestacion y healthchecks con Airflow.
7. Dashboard web con vistas de Tiempo Real y Red Logistica desacopladas.

9.4 2. Diagrama de arquitectura

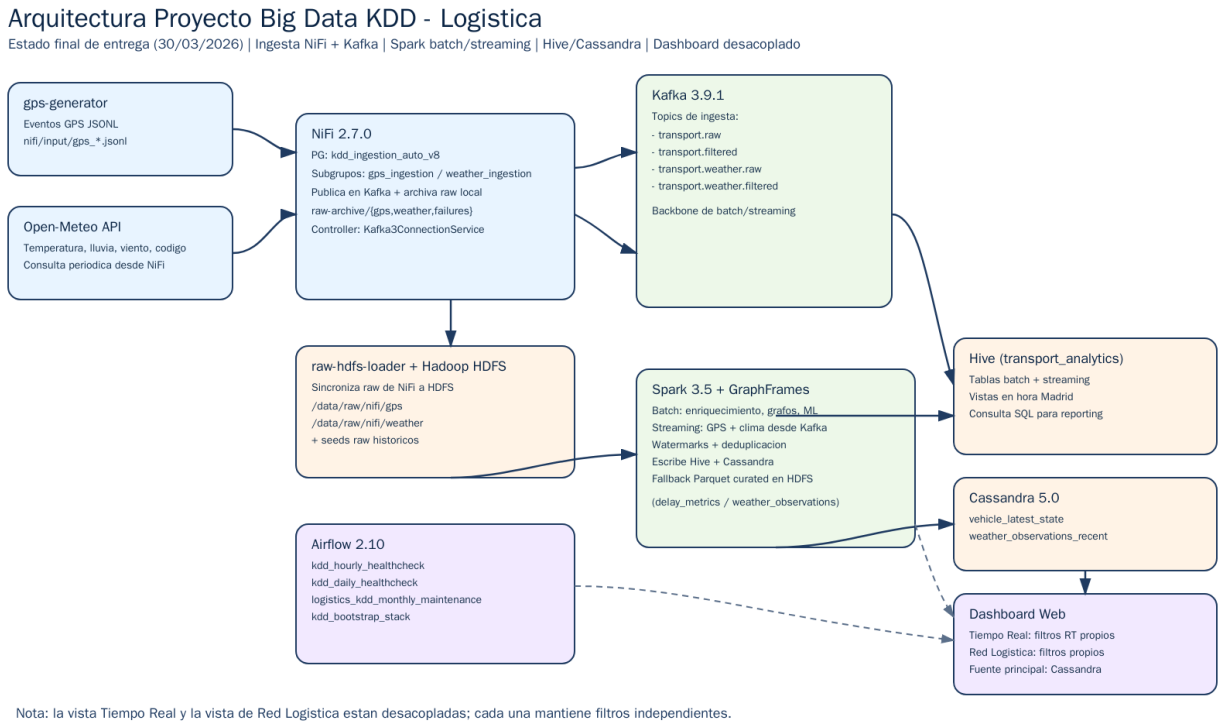

Figure 10: Diagrama de arquitectura

Archivo de imagen:

- `docs/architecture-diagram.png`

9.5 3. Componentes y responsabilidades

- `gps-generator`
  - Genera eventos GPS en `nifi/input/gps_*.jsonl`.
  - Mantiene estado de trayectos en `nifi/input/.vehicle_path_state.json`.
  - Usa rutas realistas derivadas del grafo logistico y flota de `data/master/vehicles.csv`.
- `NiFi 2.7.0`
  - PG principal: `kdd_ingestion_auto_v9`.
  - Subgrupos: `gps_ingestion`, `weather_ingestion`.
  - Publica eventos a Kafka y archiva raw local (`raw-archive`).
  - Evita sobrescrituras de GPS raw con renombrado unico por split (`filename + fragment.index + UUID`).

- `Kafka 3.9.1`
  - Topics:
    * `transport.raw`
    * `transport.filtered`
    * `transport.weather.raw`
    * `transport.weather.filtered`
- `raw-hdfs-loader` + `Hadoop HDFS`
  - Sincronizan raw local de NiFi a:
    * `/data/raw/nifi/gps`
    * `/data/raw/nifi/weather`
- `Spark 3.5 + GraphFrames`
  - Batch: enriquecimiento, agregaciones, grafos, shortest paths y ML.
  - Streaming: GPS y clima desde Kafka con watermark + deduplicacion.
  - Persistencia en Hive y Cassandra (con fallback Parquet curated), incluyendo `transport_analytics.enric`
  - Modo `insights-sync` para consolidar snapshots de insights de red en Hive.
- `Hive`
  - Almacen analitico SQL (`transport_analytics`).
  - Tablas batch + streaming y vistas en hora Madrid.
- `Cassandra`
  - Estado de baja latencia:
    * `transport.vehicle_latest_state`
    * `transport.weather_observations_recent`
    * `transport.network_insights_snapshots`
- `Airflow`
  - `kdd_hourly_healthcheck`
  - `kdd_daily_healthcheck`
  - `kdd_bootstrap_stack`
  - `logistics_kdd_monthly_maintenance`
  - El bootstrap ejecuta `scripts/ensure_hive_streaming_compat.sh` para asegurar vistas/tablas streaming.
- `Dashboard`
  - Fuente principal: Cassandra.
  - Fallback controlado a ficheros NiFi para vehiculos/clima.
  - Filtros desacoplados por vista.
  - Insights live de red y resumen de aristas con telemetria (`live_edge_summary`).
  - Motor de rutas multiobjetivo con:
    * 6 perfiles (`balanced`, `fastest`, `resilient`, `eco`, `low_risk`, `reliable`),
    * pesos `time/risk/eco`,
    * patron temporal (`auto`, `peak`, `offpeak`, `night`),
    * exclusion de nodos (`avoid_nodes`).
  - Operativa ML integrada:
    * trigger de reentreno (`/api/ml/retrain`),
    * estado/recomendacion (`/api/ml/retrain/status`),

    * persistencia de estado de recomendacion en Cassandra (`transport.model_retrain_state`).

9.6 4. Flujo de datos end-to-end

1. `gps-generator` y `Open-Meteo` alimentan NiFi.
2. NiFi publica a Kafka (raw y filtered) y deja copia raw local.
3. `raw-hdfs-loader` mueve raw a HDFS.
4. Spark streaming consume filtered y escribe Hive/Cassandra.
5. Spark batch recalcula historicos, grafos y modelo ML.
6. Dashboard consulta principalmente Cassandra para operacion.

9.7 5. Capas de almacenamiento

- `Raw (Bronze)`
  - JSON/JSONL original sin transformar.
  - Ejemplo: `/data/raw/nifi/...`
- `Curated (Silver/Gold)`
  - Parquet + tablas Hive para consulta eficiente.
  - Ejemplo: `/data/curated/...` y `transport_analytics.*`

9.8 6. Decision de desacople en dashboard

Se adopta desacople intencional para reducir ambiguedad operativa:

- `Operacion en Tiempo Real` usa filtros `Origen RT` / `Destino RT`.
- `Analisis de Red Logistica` usa `Origen` / `Destino` / `Perfil`.

Un cambio en una vista no altera la otra.

10 Release Notes - 04/04/2026

10.1 Portada

- Proyecto: `Proyecto Big Data KDD - Logistica`
- Documento: `Release notes iteracion 04/04/2026`
- Version: `v1.4-entrega`
- Fecha: `04/04/2026`
- Repositorio GitHub: `https://github.com/raulsistemasydesarrollo/KDD-Ingenieria-de-datos`

10.2 Resumen ejecutivo

Esta iteracion consolida el estado operativo final del stack con foco en:

1. Orquestacion mensual robusta en Airflow sin solapamientos.
2. Operativa de reentreno IA sincronizada entre DAG y dashboard.
3. Documentacion integral actualizada a fecha `04/04/2026`.
4. Estrategia segura para preview de ficheros en HDFS desde navegador.

10.3 Cambios funcionales

10.3.1 Airflow + DAG mensual

- `logistics_kdd_monthly_maintenance` endurecido con `max_active_runs=1`.
- Reentreno mensual encapsulado en `scripts/airflow_retrain_with_status.sh`.
- Marcador runtime compartido para exponer estado de reentreno durante ejecucion.

10.3.2 Dashboard (reentreno IA)

- Bloque anti-colision de triggers:
  - si hay reentreno activo (incluido origen Airflow), `POST /api/ml/retrain` devuelve `409`.
- Cabecera IA ampliada:
  - `Ultimo reentreno`
  - `Siguiente programado`
  - mostrado en horario `Europe/Madrid`.
- `GET /api/ml/retrain/status` ampliado con `schedule_info`.

10.3.3 HDFS UI (preview de fichero)

- Se adopta como ruta recomendada un ajuste de host local:
  - script `scripts/toggle_hdfs_browser_preview_hosts.sh`
  - objetivo: resolver `hadoop` en navegador para redirects WebHDFS.
- Se documenta explicitamente la alternativa temporal en `hdfs-site.xml` y su riesgo operativo.

10.4 Cambios de documentacion

- Portadas/versiones/fechas alineadas a `04/04/2026` en docs principales.
- Capturas operativas actualizadas:
  - `docs/airflow.png`
  - `docs/dashboard.png`
- Documentos sincronizados:
  - `README.md`
  - `docs/ENTREGA.md`
  - `docs/memoria-tecnica-sistema.md`
  - `docs/resumen-ejecutivo-memoria.md`
  - `docs/dashboard.md`
  - `docs/operations.md`
  - `docs/manual-usuario.md`
  - `docs/architecture.md`
  - `docs/nifi-flow.md`
  - `docs/nifi-weather-template.md`
  - `docs/CODE-ANNOTATION-GUIDE.md`

10.5 Implicaciones operativas (HDFS preview)

Camino recomendado:

```
./scripts/toggle_hdfs_browser_preview_hosts.sh status
sudo ./scripts/toggle_hdfs_browser_preview_hosts.sh enable
```

Reversion:

```
sudo ./scripts/toggle_hdfs_browser_preview_hosts.sh disable
```

Riesgo de la alternativa temporal sobre `hdfs-site.xml`:

- Aunque puede facilitar preview web, puede romper escrituras HDFS desde Spark/Airflow en contenedores:
  - `Connection refused`
  - `could only be written to 0 of the 1 minReplication nodes`

10.6 Referencias rapidas

- Guia operativa: `docs/operations.md`
- Manual de usuario: `docs/manual-usuario.md`
- Dashboard: `docs/dashboard.md`
- Indice de entrega: `docs/ENTREGA.md`

11 Proyecto Big Data KDD - Java 21 + Docker

Implementacion de una plataforma Big Data orientada a logistica y transporte siguiendo el ciclo KDD del enunciado, usando Java 21 para la aplicacion Spark y una arquitectura desplegada con contenedores Docker.

Repositorio GitHub oficial del proyecto:

- `https://github.com/raulsistemasydesarrollo/KDD-Ingenieria-de-datos`

11.1 Arquitectura

- `NiFi 2.7.0`: ingesta desde API/logs simulados y publicacion en Kafka.
- `Kafka 3.9.1 (KRaft)`: bus de eventos para crudo y filtrado.
- `gps-generator`: generador continuo de logs GPS para alimentar NiFi (`nifi/input`).
- `raw-hdfs-loader`: sincronizacion continua de raw archivado por NiFi hacia HDFS (`/data/raw/nifi`).
- `Hadoop 3.4.2 pseudodistribuido`: HDFS + YARN en un unico nodo.
- `Hive`: almacenamiento SQL historico y catalogo.
- `Cassandra 5.0`: ultimo estado por vehiculo para consultas de baja latencia.
- `Spark 3.5.x + GraphFrames`: limpieza, enriquecimiento, streaming, analisis de grafos y modelo ML de riesgo de retraso.
- `Airflow 2.10.x`: orquestacion del mantenimiento mensual.

- `Dashboard Web`: visualizacion en tiempo real de vehiculos (mapa), trazas de movimiento y analitica de rutas con grafo + impacto meteorologico.

11.2 Estructura

- `docker-compose.yml`: stack completo.
- `docker/`: Dockerfiles y configuracion de infraestructura.
- `spark-app/`: proyecto Maven en Java 21 para Spark.
- `dags/`: DAGs de Airflow.
- `data/`: datasets semilla para demo.
- `nifi/`: material para la ingesta y notas del flujo.
- `docs/`: documentacion tecnica y pasos operativos.
- `dashboard/`: servidor web ligero + frontend del dashboard logistico.

Indice maestro de entrega:

- docs/ENTREGA.md
- docs/manual-usuario.md (manual de usuario completo)
- docs/architecture.md (incluye diagrama docs/architecture-diagram.png)
- CHANGELOG.md (historial consolidado de cambios)

11.3 Arranque rapido

1. Construir y levantar la plataforma:

   ```
   docker compose up --build -d
   ```

2. Compilar la aplicacion Spark:

   ```
   docker compose exec spark-client mvn -f /opt/spark-app/pom.xml -DskipTests package
   ```

3. Cargar datasets de demo en HDFS:

   ```
   docker compose exec hadoop bash /opt/hadoop/bootstrap/load-seed-data.sh
   ```

4. Ejecutar el job batch/grafo:

   ```
   docker compose exec spark-client bash /opt/spark-app/run-batch.sh
   ```

   Opcional (sincronizar snapshots de insights Cassandra -> Hive):

   ```
   sg docker -c "docker compose exec -T spark-client /opt/spark-app/run-insights-sync.sh"
   ```

5. Ejecutar el job de streaming:

   ```
   docker compose exec spark-client bash /opt/spark-app/run-streaming.sh
   ```

   Opcional (forzar reproceso desde el inicio de los topics):

   ```
   docker compose exec -e STREAMING_STARTING_OFFSETS=earliest spark-client bash /opt/spark-app
   ```

   Opcional (zona horaria SQL de Spark; por defecto `Europe/Madrid`):

```
docker compose exec -e SPARK_SQL_TIMEZONE=Europe/Madrid spark-client bash /opt/spark-app/ru
```

6. Configurar y arrancar flujo NiFi automaticamente (sin UI manual):

   ```
   ./scripts/bootstrap_nifi_flow.sh
   ```

   El bootstrap usa por defecto `NIFI_PG_NAME=kdd_ingestion_auto_v9` y crea dos subgrupos:

   - `gps_ingestion`
   - `weather_ingestion`

7. Abrir dashboard logistico (mapa + grafos + meteo):

   ```
   docker compose up -d dashboard
   ```

   URL:

   - `http://localhost:8501`

   Comportamiento actual de filtros:

   - Vista Tiempo Real (izquierda): usa `Origen RT` / `Destino RT` y afecta solo esa vista.
   - Vista Analisis Logistico (derecha): usa `Origen` / `Destino` y afecta solo mapa/logistica/tablas de esa columna.
   - Ambas vistas estan desacopladas intencionalmente.

   Capacidades actuales de routing/analitica en dashboard:

   - Perfiles: `balanced`, `fastest`, `resilient`, `eco`, `low_risk`, `reliable`.
   - Multiobjetivo: pesos `tiempo/riesgo/eco` (normalizados).
   - Patron horario: `auto`, `peak`, `offpeak`, `night`.
   - Exclusiones de nodo: `Evitar nodos`.
   - Candidatas de ruta: hasta 4 alternativas con `delta_vs_best`.
   - Reentreno IA:
     - trigger `POST /api/ml/retrain`
     - estado/recomendacion `GET /api/ml/retrain/status`

11.4 Scripts operativos

- Arranque completo del stack:

  ```
  ./scripts/start_kdd.sh
  ```

  Incluye automaticamente:

  - bootstrap de flujo NiFi,
  - `scripts/ensure_hive_streaming_compat.sh`,
  - sanity check Hive (`SHOW TABLES` + `COUNT(*)` sobre vistas Madrid),
  - repoblado automatico de meteo en Hive desde Cassandra si `weather_rows=0`.

- Parada completa del stack:

  ```
  ./scripts/stop_kdd.sh
  ```

- Limpieza de estado streaming (tablas Hive + checkpoints HDFS):

  ```
  ./scripts/reset_streaming_state.sh
  ```

- Reset de demo (limpieza de eventos locales + reset streaming + recrear generadores):

  ```
  ./scripts/reset_demo_data.sh
  ```

  Hard reset (incluye limpieza de `/data/raw/nifi` en HDFS):

  ```
  ./scripts/reset_demo_data.sh --hard
  ```

11.5 Operacion diaria (checks, reinicios, BBDD y troubleshooting)

Preview HDFS en navegador (opcional):

```
./scripts/toggle_hdfs_browser_preview_hosts.sh status
sudo ./scripts/toggle_hdfs_browser_preview_hosts.sh enable
```

Revertir ajuste:

```
sudo ./scripts/toggle_hdfs_browser_preview_hosts.sh disable
```

Implicaciones:

- Este ajuste solo toca `/etc/hosts` del host (`127.0.0.1 hadoop`).
- Permite que el navegador siga el redirect de NameNode a `http://hadoop:9864/...`.
- No modifica `hdfs-site.xml`, por tanto **no rompe Spark** ni clientes internos Docker.

Cambio temporal alternativo (NO recomendado salvo prueba puntual):

1. Editar `docker/hadoop/conf/hdfs-site.xml` y fijar:
   - `dfs.datanode.hostname=localhost`
   - `dfs.client.use.datanode.hostname=true`
   - `dfs.datanode.use.datanode.hostname=true`
2. Reconstruir/recrear `hadoop`.

Riesgo/impacto del cambio temporal alternativo:

- El preview web desde host suele funcionar.
- Clientes internos (Spark/Airflow en contenedores) pueden fallar escribiendo en HDFS con `Connection refused` o `could only be written to 0 of the 1 minReplication nodes`.
- Tras la prueba, revertir esos 3 parametros y recrear `hadoop`.

Checks de salud rapidos:

```
docker compose ps
curl -fsS http://localhost:8501/health
curl -fsS http://localhost:8080/health
docker compose exec -T kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 --list
```

Reinicio de servicios frecuentes:

```
docker compose restart dashboard
docker compose restart nifi
docker compose restart spark-client
```

Consultas BBDD desde terminal:

```
docker compose exec -T spark-client spark-sql -e "SHOW TABLES IN transport_analytics;"
docker compose exec -T spark-client spark-sql -e "SELECT COUNT(*) AS weather_rows FROM transport
docker compose exec -T cassandra cqlsh -e "SELECT vehicle_id, warehouse_id, route_id, last_event
```

Acciones de recuperacion tipicas:

```
./scripts/ensure_hive_streaming_compat.sh
./scripts/repopulate_hive_weather_from_cassandra.sh
./scripts/reset_streaming_state.sh
./scripts/cleanup_airflow_failed_runs.sh --apply
./scripts/bootstrap_nifi_flow.sh
```

Guia operativa completa:

- `docs/operations.md`

11.6 Validacion automatica Hive

Para validar en un solo comando la escritura batch + streaming en Hive:

```
chmod +x scripts/validate_hive_pipeline.sh
./scripts/validate_hive_pipeline.sh
```

Documentacion detallada:

- `docs/hive-validation.md`

Para limpiar tablas/checkpoints de streaming (quitar estado de pruebas):

```
chmod +x scripts/reset_streaming_state.sh
./scripts/reset_streaming_state.sh
```

Validacion especifica del fix de parseo meteo (numeros y numeros-string):

```
./scripts/validate_weather_parsing_fix.sh
```

Garantizar compatibilidad Hive de objetos streaming (tablas/vistas Madrid):

```
./scripts/ensure_hive_streaming_compat.sh
```

Repoblar Hive meteo desde Cassandra (cuando `v_weather_observations_madrid` esta vacia):

```
./scripts/repopulate_hive_weather_from_cassandra.sh
```

11.7 Formato de datos (raw vs procesado)

Se aplica la separacion recomendada:

- `Raw`: eventos sin transformar en `JSON/JSONL`.
- `Procesado`: datasets limpios y agregados en `Parquet` (consultables via Hive).

Referencias del proyecto:

- Raw GPS base Spark batch: `hdfs://hadoop:9000/data/raw/gps_events.jsonl`
- Raw NiFi archivado en host: `nifi/raw-archive/`
- Raw NiFi sincronizado a HDFS: `/data/raw/nifi`
- Curated batch en Parquet: `hdfs://hadoop:9000/data/curated/enriched_events`
- Curated streaming fallback Parquet:
  - `hdfs://hadoop:9000/data/curated/delay_metrics_streaming`
  - `hdfs://hadoop:9000/data/curated/weather_observations_streaming`
  - `hdfs://hadoop:9000/data/curated/enriched_events_streaming`

Comprobacion rapida:

```
sg docker -c "docker compose exec -T hadoop hdfs dfs -ls -R /data/raw | head -n 40"
sg docker -c "docker compose exec -T hadoop hdfs dfs -ls -R /data/curated | head -n 80"
```

11.7.1 Matriz operativa (resumen)

Flujo	Kafka	HDFS	Hive
GPS raw	`transport.raw`	`/data/raw/nifi/gps/...`	No aplica
GPS filtrado	`transport.filtered`	`/data/curated/delay_metrics_streaming`	`transport_analytics.delay_metri`
GPS enriquecido streaming	`transport.filtered`	`/data/curated/enriched_events_streaming`	`transport_analytics.enriched_ev`
Weather raw	`transport.weather.raw`	`/data/raw/nifi/weather`	No aplica
Weather filtrado	`transport.weather.filtered`	`/data/curated/weather_observations_streaming`	`transport_analytics.weather_o` (vista operativa)

Batch historico GPS	No aplica	`/data/curated/enriched_events`	`transport_analytics.enriched_ev`
			`transport_analytics.delay_metri`
			`transport_analytics.route_graph`

Detalle operativo completo:

- `docs/operations.md` (seccion `Matriz operativa end-to-end`)

11.8 Vistas Hive en hora Madrid

Se han creado vistas para consultar timestamps en `Europe/Madrid` sin conversion manual:

- `transport_analytics.v_weather_observations_madrid`
- `transport_analytics.v_delay_metrics_streaming_madrid`

Ejemplos:

```sql
SELECT weather_event_id, weather_timestamp_utc, weather_timestamp_madrid
FROM transport_analytics.v_weather_observations_madrid
ORDER BY weather_timestamp_utc DESC
LIMIT 10;
```

```sql
SELECT warehouse_id, window_start_utc, window_start_madrid, window_end_utc, window_end_madrid
FROM transport_analytics.v_delay_metrics_streaming_madrid
ORDER BY window_start_utc DESC
LIMIT 10;
```

Nota de compatibilidad:

- Si el estado streaming fue limpiado, el proyecto recrea objetos base con `scripts/ensure_hive_streaming_comp`
- Esto evita errores `TABLE_OR_VIEW_NOT_FOUND` en las vistas Madrid aunque no haya datos todavia.

11.9 Servicios

- NiFi: `http://localhost:8443/nifi`
- Kafka broker interno: `kafka:9092`
- Hadoop/YARN: `http://localhost:9870`, `http://localhost:8088`
- HiveServer2: `localhost:10000`
- Cassandra CQL: `localhost:9042`
- Airflow: `http://localhost:8080`
- Dashboard logistico: `http://localhost:8501`

Usuario NiFi:

- usuario: `admin`
- password: `adminadminadmin`

Usuario por defecto Airflow:

- usuario: `airflow`
- password: `airflow`

DAGs relevantes:

- `logistics_kdd_monthly_maintenance` (mantenimiento mensual)
- `kdd_daily_healthcheck` (checklist diario automatizado)
- `kdd_hourly_healthcheck` (checklist horario automatizado)
- `kdd_bootstrap_stack` (arranque del stack no-Airflow desde Airflow)

Auto-recuperacion meteo en DAGs de healthcheck:

- `kdd_daily_healthcheck` y `kdd_hourly_healthcheck` incluyen la tarea `repopulate_hive_weather_if_empty`
- Si `SELECT COUNT(*) FROM transport_analytics.v_weather_observations_madrid` devuelve `0`, ejecutan `./scripts/repopulate_hive_weather_from_cassandra.sh` antes del smoke query meteo.
- Si hay filas, no repueblan.

Alertas Airflow:

- Todos los DAGs incorporan callback de fallo.
- Para email en fallo: exporta `AIRFLOW_ALERT_EMAIL` en el entorno de Airflow.

Arranque "Airflow primero y despues stack desde DAG":

```
./scripts/start_airflow_then_stack.sh
```

Limpieza de runs `failed` historicos de healthchecks (modo simulacion + apply):

```
./scripts/cleanup_airflow_failed_runs.sh
./scripts/cleanup_airflow_failed_runs.sh --apply
```

Limpieza de Process Groups legacy de NiFi (manteniendo `kdd_ingestion_auto_v9`):

```
./scripts/cleanup_nifi_legacy_pgs.py
```

11.10 Cambios recientes (03/04/2026)

- Entrega documental v1.3 sincronizada en todos los documentos principales de `docs/`:
  - portada, version y fecha de entrega alineadas a `03/04/2026`.
- Captura del dashboard actualizada y referenciada en documentacion:
  - `docs/dashboard.png`,
  - `docs/dashboard.md`,
  - `docs/memoria-tecnica-sistema.md`.
- Nuevo documento de release notes para la iteracion actual:
  - `docs/release-notes-2026-04-04.md`.
- Estado funcional del dashboard alineado con codigo actual:
  - perfiles adicionales `eco`, `low_risk`, `reliable`,
  - patron horario y pesos `tiempo/riesgo/eco`,
  - panel `Evitar nodos`,
  - boton `Reentrenar IA` + recomendacion de deriva,
  - bloque de modelos IA en cabecera con dos paneles:
    - panel izquierdo: `EN USO` + candidato elegido (A/B/C) y comparativa RMSE,
    - panel derecho: descripcion de candidatos en columna unica,
  - rutas de Tiempo Real muestran ruta completa (`origen -> intermedios -> final`) cuando el generador publica `planned_route_nodes`,
  - tabla de vehiculos y panel de historial muestran `ETA nodos restantes` y `ETA destino final`.
- ML de riesgo de retraso reforzado con validacion comparativa en cada batch:
  - candidatos `baseline_rf`, `tuned_baseline_rf`, `enhanced_rf`,
  - `enhanced_rf` incorpora features de clima y congestion alineadas por ventana temporal de 15 minutos y almacen,
  - seleccion automatica del mejor por RMSE para evitar regresiones,
  - benchmark actual (dataset semilla 20k): `baseline_rmse=6.1329`, `tuned_baseline_rmse=6.1343`, `enhanced_rmse=6.0206` -> seleccionado `enhanced_rf`.

- Modelo en HDFS validado tras reentreno:
  - ruta `hdfs://hadoop:9000/models/delay_risk_rf`,
  - tamano aproximado `1.2 MB` (Spark ML pipeline serializado en Parquet/Snappy).
- Builders de PDF de entrega ajustados a la iteracion activa:
  - `scripts/build_delivery_pdf.py` apunta a release notes actual,
  - `scripts/build_delivery_pdf_professional.sh` genera nombre de salida con fecha dinamica.

11.11 Cambios recientes (04/04/2026)

- Capturas de estado actualizadas:
  - `docs/airflow.png` (vista Airflow renovada),
  - `docs/dashboard.png` (vista dashboard renovada).
- Reentreno IA y orquestacion:
  - DAG `logistics_kdd_monthly_maintenance` endurecido con `max_active_runs=1` para evitar runs paralelos.
  - Reentreno mensual via `scripts/airflow_retrain_with_status.sh` con marcador runtime compartido.
  - Dashboard bloquea trigger manual cuando detecta reentreno externo en curso (`409 Conflict`).
- Cabecera de IA en dashboard:
  - muestra `Ultimo reentreno` y `Siguiente programado` sobre el boton `Reentrenar IA`.
  - horario de referencia mostrado en `Europe/Madrid`.
  - endpoint `GET /api/ml/retrain/status` amplia respuesta con `schedule_info`.
- Operativa HDFS UI:
  - se documenta ajuste seguro por `/etc/hosts` para preview de ficheros desde navegador sin afectar Spark.
- Release notes de esta iteracion:
  - `docs/release-notes-2026-04-04.md`.

11.12 Cambios recientes (31/03/2026)

- Dashboard:
  - Etiquetas de nodos en mapa de Tiempo Real normalizadas a codigo de 3 letras (igual que el mapa logistico).
  - Etiquetas de nodos de Tiempo Real con transparencia para no tapar vehiculos.
  - Selectores `Origen/Destino` (RT y Red Logistica) ordenados alfabeticamente.
  - Todas las tablas del dashboard ordenables por columna (click en cabeceras).
  - Tabla bajo mapa logistico enfocada en origen/destino seleccionados (o global en `TODOS`).
  - Nuevos insights live de red (cuellos de botella, nodos criticos e historico).
  - Resolucion de congestion `UNKNOWN` a `low/medium/high` cuando no hay muestras live.
- Backend dashboard / Spark:
  - Nuevos campos live por arista (`effective_avg_delay_minutes`, `live_avg_delay_minutes`, `live_sample_count`, `congestion_level`) y resumen `live_edge_summary`.

  - Persistencia de snapshots de insights en Cassandra (`transport.network_insights_snapshots`).
  - Endpoint historico de insights: `/api/network/insights/history`.
  - Nuevo modo Spark `insights-sync` + launcher `spark-app/run-insights-sync.sh` para consolidar en Hive:
    * `transport_analytics.network_insights_snapshots_hive`
    * `transport_analytics.network_insights_hourly_trends`
- Red y simulacion:
  - Red actual ampliada a 15 nodos y flota de 15 vehiculos (14 activos + 1 en mantenimiento).
  - Script de regeneracion de aristas por cercania geografica:
    * `python3 scripts/rebuild_graph_edges_by_proximity.py --k 4`
  - Reglas de negocio aplicadas en la red:
    * prohibidas directas: `BCN-MUR`, `ACO-BIO`, `VAL-ALM`
    * obligatorias: `BCN-VAL`, `VAL-MUR`, `ACO-GIJ`, `GIJ-BIO`, `MAD-SEV`, `MAD-MAL`
    * corredor sur sin atajos: `CAC-SEV-MAL-ALM-MUR`
- Operativa:
  - Si aparece `permission denied` al lanzar `run-insights-sync.sh`, ejecutar:
    * `chmod +x spark-app/run-insights-sync.sh`
    * reconstruir/recrear `spark-client` para refrescar permisos en imagen/contenedor.

11.13 Cambios recientes (30/03/2026)

- Airflow:
  - Corregidas `smoke_query_weather` y `smoke_query_delay` en healthchecks diario/horario para evitar caida cuando faltan tablas streaming fuente.
  - Runs historicos `failed` limpiados para `kdd_daily_healthcheck` y `kdd_hourly_healthcheck`.
  - Auto-recuperacion de meteo Hive en healthchecks (`repopulate_hive_weather_if_empty`) cuando `v_weather_observations_madrid` queda en `0` filas.
- NiFi:
  - Nuevo bootstrap visual por subflujos: `gps_ingestion` y `weather_ingestion`.
  - Proceso de limpieza legacy disponible en `scripts/cleanup_nifi_legacy_pgs.py`.
- Arranque:
  - `scripts/start_kdd.sh` y `scripts/bootstrap_nifi_flow.sh` apuntan por defecto a `kdd_ingestion_auto_v9`.
  - `scripts/start_kdd.sh` repuebla meteo en Hive desde Cassandra si la vista Madrid esta vacia.
- Spark + Dashboard:
  - El dashboard usa Cassandra como fuente primaria de clima (`transport.weather_observations_recent`) con fallback a `nifi/raw-archive/weather`.
  - `LogisticsAnalyticsJob` en modo streaming persiste observaciones meteo recientes en Cassandra para mejorar precision de ruta y panel de clima.
  - `LogisticsAnalyticsJob` en modo streaming persiste tambien `transport_analytics.enriched_events_` en Hive (ademas de `delay_metrics_streaming` y `weather_observations_streaming`).
  - `spark-app/run-streaming.sh` y `spark-app/run-batch.sh` recompilan el JAR automatica-

mente si hay cambios en `src/` o `pom.xml`.
  - Ajuste de ETA en tiempo real: resincronizacion inmediata cuando hay divergencia grande entre ETA cacheado y ETA fisico (distancia/velocidad) o cambio de destino estimado.
- GPS/NiFi:
  - El generador GPS usa rutas realistas derivadas del grafo logistico y ya no etiquetas aleatorias fijas.
  - Flota leida desde `data/master/vehicles.csv` (IDs actuales `TRUCK-*`; filtra por `status` activo).
  - En NiFi se evita sobrescritura de raw GPS: cada split se archiva con nombre unico (`${filename}_${fragment.index}_${UUID()}.jsonl`).

11.14 Notas

- La arquitectura Hadoop es de nodo unico en modo pseudodistribuido.
- El proyecto prioriza reproducibilidad local y separa claramente batch, streaming y orquestacion.
- Se contempla ingesta meteo (API publica) hacia `transport.weather.raw` y `transport.weather.filtered`, con persistencia en Hive.
- Hay documentacion operativa en `docs/architecture.md`, `docs/nifi-flow.md`, `docs/nifi-weather-templat` y `docs/hive-validation.md`.
- Guia operativa consolidada: `docs/operations.md`.
- Guia de dashboard y leyenda visual: `docs/dashboard.md`.
- Endpoint debug de fuentes del dashboard: `GET /api/debug/sources`.
- Plan TODO de cumplimiento del enunciado por fases KDD: `docs/kdd-todo.md`.
- Bitacora de cambios de la iteracion 04/04/2026: `docs/release-notes-2026-04-04.md`.
- Memoria tecnica detallada del sistema (entrega): `docs/memoria-tecnica-sistema.md`.
- Resumen ejecutivo (1-2 paginas) para portada/anexo inicial: `docs/resumen-ejecutivo-memoria.md`.

12 Changelog

Todos los cambios relevantes de este proyecto se documentan en este archivo.

El formato sigue Keep a Changelog.

12.1 [Unreleased]

12.1.1 Added

- Nuevo manual de usuario integral:
  - `docs/manual-usuario.md`
  - incluye arranque/parada del stack, uso completo del dashboard, uso de DAGs de Airflow, checks de salud, consultas BBDD y troubleshooting.
- Persistencia streaming en Hive de eventos enriquecidos: `transport_analytics.enriched_events_streaming`.
- Nuevo fallback Parquet de streaming: `/data/curated/enriched_events_streaming`.
- Persistencia de snapshots de insights en Cassandra: `transport.network_insights_snapshots`.
- Nuevo endpoint de historico de insights: `/api/network/insights/history`.

- Nuevo launcher Spark: `spark-app/run-insights-sync.sh`.
- Nuevas tablas Hive de consolidacion de insights:
  - `transport_analytics.network_insights_snapshots_hive`
  - `transport_analytics.network_insights_hourly_trends`
- Script `scripts/rebuild_graph_edges_by_proximity.py` para regenerar aristas por cercania geografica con reglas de negocio.

12.1.2 Changed

- Documentacion de entrega sincronizada a `v1.3` (`03/04/2026`) en README y documentos principales de `docs/`.
- Nuevo release notes de iteracion actual: `docs/release-notes-2026-04-03.md`.
- `docs/operations.md` ampliado con catalogo operativo de scripts:
  - checks de salud,
  - reinicio completo/parcial de servicios,
  - consultas BBDD (Hive/Cassandra),
  - troubleshooting y recuperacion.
- `README.md` ampliado con bloque de operacion diaria (healthchecks, reinicios, consultas y recuperacion).
- Enlaces al nuevo manual de usuario añadidos en documentacion principal:
  - `README.md`
  - `docs/ENTREGA.md`
  - `docs/operations.md`
  - `docs/dashboard.md`
- Builders PDF actualizados para consumir release notes actual:
  - `scripts/build_delivery_pdf.py`
  - `scripts/build_delivery_pdf_professional.sh`
- Generador GPS actualizado a rutas realistas sobre grafo logistico y flota actual leida desde `data/master/vehicles.csv`.
- Bootstrap NiFi actualizado a `kdd_ingestion_auto_v9`.
- Flujo GPS NiFi actualizado con renombrado unico por split (`${filename}_${fragment.index}_${UUID()}.js` para evitar sobrescrituras en `raw-archive/gps`.
- Parseo de timestamps ISO UTC robusto en Spark streaming para GPS y clima.
- Documentacion tecnica y operativa actualizada para reflejar el estado real de `v9` y las nuevas salidas streaming.
- Dashboard actualizado:
  - tablas ordenables por columna,
  - selectores de origen/destino ordenados alfabeticamente,
  - etiquetas de nodos RT en codigo de 3 letras y semitransparentes.
- Red logistico-geografica consolidada:
  - 15 nodos (`ACO, ALM, BCN, BIO, CAC, GIJ, LIS, MAD, MAL, MUR, OPO, SEV, VAL, VLL, ZAR`),
  - 15 vehiculos (`14` activos y `1` en mantenimiento),

  - restricciones de conectividad por corredor y atajos prohibidos.
- Actualizacion integral de documentacion de entrega a `v1.2` (`02/04/2026`) en los documentos principales de `docs/`.
- Captura de dashboard renovada en `docs/dashboard.png` y leyenda sincronizada en documentacion funcional/tecnica.
- `scripts/build_delivery_pdf.py` actualizado para incluir `docs/release-notes-2026-04-02.md`.
- `scripts/build_delivery_pdf_professional.sh` preparado para salida PDF con fecha dinamica.
- Dashboard de rutas ampliado con nuevos perfiles `low_risk` y `reliable` (ademas de `balanced`, `fastest`, `resilient`, `eco`).
- Dashboard actualizado con optimizacion multiobjetivo (`objective_time`, `objective_risk`, `objective_eco`), `temporal_mode` y exclusion de nodos (`avoid_nodes`).
- Nuevos endpoints ML operativos en dashboard:
  - `POST /api/ml/retrain`
  - `GET /api/ml/retrain/status`
- Persistencia del estado/recomendacion de reentreno en Cassandra (`transport.model_retrain_state`).

12.1.3 Fixed

- Correccion de sobrescritura de ficheros GPS en `nifi/raw-archive/gps` al archivar flowfiles de `SplitText`.
- Correccion de `UNKNOWN` en congestion de insights cuando no hay muestras live (normalizacion a `low/medium/high`).
- Correccion en computo de insights por perfil (`KeyError: profile_minutes_sum`).

12.1.4 Removed

- Sin cambios todavia.

12.2 [1.0.0] - 2026-03-30

12.2.1 Added

- Script `scripts/repopulate_hive_weather_from_cassandra.sh` para repoblar `transport_analytics.weath` desde Cassandra (`transport.weather_observations_recent`).
- Tarea `repopulate_hive_weather_if_empty` en los DAGs:
  - `dags/kdd_hourly_healthcheck_dag.py`
  - `dags/kdd_daily_healthcheck_dag.py`

12.2.2 Changed

- `scripts/start_kdd.sh` incluye repoblado automatico de meteo en Hive cuando `v_weather_observations_mad` devuelve `weather_rows=0`.
- `docs/operations.md` actualizado con flujo de diagnostico y repoblado manual.
- `README.md` actualizado con el flujo de auto-recuperacion meteo.

12.2.3 Fixed

- ETA incoherente en dashboard por cache/suavizado:
  - resincronizacion forzada al cambiar destino estimado,
  - resincronizacion forzada ante divergencia grande entre ETA cacheado y ETA fisico (`distancia/velocidad + delay`).
  - archivo afectado: `dashboard/static/app.js`.

12.2.4 Removed

- Sin cambios.

12.2.5 Documentation

- Normalizacion de enlaces en `docs/CODE-ANNOTATION-GUIDE.md` a rutas relativas.
- Actualizacion de:
  - `docs/dashboard.md` (logica ETA y estabilidad),
  - `docs/release-notes-2026-04-02.md`,
  - `docs/operations.md` (troubleshooting ETA y Hive meteo),
  - `README.md` (operacion consolidada).